

Unit 4

PROCESSOR FUNDAMENTALS

48 items · 357 marks

Syllabus sections in this unit

- 4.1 Central Processing Unit (CPU) Architecture
- 4.2 Assembly Language
- 4.3 Bit manipulation

Unit 4: Processor Fundamentals

1. M/J 26/P11/Q3/a,b,c

- 3 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

instruction		explanation
opcode	operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
STO	<address>	Store the contents of ACC at the given address
ADD	#n / Bn / &n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
END		Return control to the operating system
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current contents of memory are:

address	data
100	200
101	205
102	100
103	50
...	
200	101
201	100
202	20
203	13
204	220
205	30

- (a) Complete the table to identify the data that will be stored in the accumulator after each set of instructions has been executed.

accumulator current data	set of instructions	new accumulator data
B11001010	ADD B00000010 INC ACC	
#100	LDM #200 ADD 200	
#103	LDD 103 DEC ACC DEC ACC	

[3]

(b) The instruction set also contains the following instructions:

instruction		explanation
opcode	operand	
AND	#n / Bn / &n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n / Bn / &n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n / Bn / &n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
CSL	#n	Bits in ACC are shifted n places to the left using a cyclic shift
CSR	#n	Bits in ACC are shifted n places to the right using a cyclic shift
ASL	#n	Bits in ACC are shifted n places to the left using an arithmetic shift
ASR	#n	Bits in ACC are shifted n places to the right using an arithmetic shift
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

- (i) Write the bit manipulation instruction that will set the ACC to $\&0$

[2]

- (ii) The ACC currently contains the following binary value.

0	0	1	1	0	1	0	1
---	---	---	---	---	---	---	---

Write the result after the instruction `LSR #3` is run.

--	--	--	--	--	--	--	--

[1]

- (c) Immediate, relative and direct are three modes of addressing used in assembly language programming.

Identify and describe **one** additional mode of addressing used in assembly language programming.

Include an example of an instruction that uses your selected mode of addressing in your answer.

mode of addressing

description with example

[3]

2. M/J 26/P13/Q7/a,b,c

7 A program is written in assembly language.

(a) Complete the table by writing the name of each mode of addressing.

mode of addressing	description
	The operand is an address where the data is stored.
	The operand is an address that stores the address where the data is stored.
	The operand is an address. The address where the data is stored is calculated by adding the operand to the content of the index register.
	The operand is the data to be used.

[4]

(b) Explain how bit manipulation can be used to monitor or control a device.

[3]

(c) Identify **one** similarity and **one** difference between a right logical shift and a right arithmetic shift.

similarity

difference

[2]

3. O/N 25/P11/Q4/a,b,c,d

4 The table shows part of the instruction set for a processor. The processor has one register, the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n / Bn / &n	Bitwise AND operation of the contents of the ACC with the operand
AND	<address>	Bitwise AND operation of the contents of the ACC with the contents of <address>
XOR	#n / Bn / &n	Bitwise XOR operation of the contents of the ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of the ACC with the contents of <address>
OR	#n / Bn / &n	Bitwise OR operation of the contents of the ACC with the operand
OR	<address>	Bitwise OR operation of the contents of the ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end.
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end.
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127 B denotes a binary number, e.g. B10010001 & denotes a hexadecimal number, e.g. &4A		

(a) The ACC currently contains the following positive binary integer:

0	0	0	1	1	1	1	0
---	---	---	---	---	---	---	---

Write a bit manipulation instruction that uses a binary shift to change the contents of the ACC to:

0	1	1	1	1	0	0	0
---	---	---	---	---	---	---	---

Instruction

[1]

(b) The ACC currently contains the following positive binary integer:

1	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---

Write the contents of the ACC after the instruction `XOR &12` is carried out.

--	--	--	--	--	--	--	--

[1]

(c) The ACC currently contains the following positive binary integer:

1	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---

Write the contents of the ACC after the instruction `AND #63` is carried out.

--	--	--	--	--	--	--	--

[1]

(d) The ACC currently contains the following positive binary integer:

1	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---

The current contents of memory are:

Address	Data
98	00100100
99	00110001
100	00110011
101	10100011
102	10101100

Write the contents of the ACC after the instruction `OR 100` is carried out.

--	--	--	--	--	--	--	--

[1]

4. O/N 25/P11/Q6/a,b

- 6 (a) The processor uses several registers, including the Accumulator (ACC) and the Current Instruction Register (CIR).

Complete the table by describing the role of each register.

Register	Role
ACC	
CIR	

[2]

- (b) Increasing the number of cores in a processor can affect the performance of a computer.

Describe the drawbacks of increasing the number of cores in a processor.

[2]

5. O/N 25/P12/Q3/a,b,c

- 3 (a) (i) State what is meant by relative addressing.

[1]

- (ii) Registers such as the Accumulator (ACC) and the Index Register (IX) are used in the CPU.

Identify **two** special purpose registers used in the CPU. Do **not** include the ACC or IX in your answers.

1

2

[2]

- (b) The following table shows part of the instruction set for a processor. The processor has two registers: the ACC and an IX.

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127		

The current contents of the main memory and the index register are shown.

Address	Instruction
98	8
99	16
100	3
101	98
102	32
IX	2

Write the contents of the ACC after each instruction is executed.

Instruction	Value in ACC
LDM #98	
LDI 101	
LDX 100	

[3]

- (c) A student buys a new computer. The table shows the specifications of the old computer and the new computer.

Old computer	New computer
1.8 GHz dual core processor	2.3 GHz dual core processor
16 MB cache	32 MB cache

Explain why increasing the clock speed **and** increasing the cache memory will improve the performance of the computer.

Clock speed

Cache memory

[4]

6. O/N 25/P13/Q2/c

- (c) A three-place arithmetic shift to the right is performed on the following two's complement negative integer.

Show the result of this arithmetic shift.

10010011

[1]

7. O/N 25/P13/Q6/a,b,c

- 6 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to the ACC
AND	#n / Bn / &n	Bitwise AND operation of the contents of the ACC with the operand
AND	<address>	Bitwise AND operation of the contents of the ACC with the contents of <address>
XOR	#n / Bn / &n	Bitwise XOR operation of the contents of the ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of the ACC with the contents of <address>
OR	#n / Bn / &n	Bitwise OR operation of the contents of the ACC with the operand
OR	<address>	Bitwise OR operation of the contents of the ACC with the contents of <address>
CMP	#n	Compare the contents of the ACC with number n
CMP	<address>	Compare the contents of the ACC with the contents of <address>
LSL	#n	Bits in the ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in the ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127 B denotes a binary number, e.g. B10010001 & denotes a hexadecimal number, e.g. &4A		

The current contents of main memory are shown:

Address	Data
100	0000 0011
101	1010 1110
102	1100 1100
103	1111 1111
104	1100 1100

(a) Complete the table by writing the contents of the ACC after the execution of each instruction.

Current contents of the ACC	Instruction	Contents of the ACC after the execution of the instruction
0000 1111	AND 101	
0000 0000	LDM #100	
0000 0001	XOR &F1	
0001 0001	CMP 101	

[4]

(b) The Von Neumann model for a computer system uses registers.

Describe the role of the Memory Address Register (MAR) and Memory Data Register (MDR) in the fetch-execute (F-E) cycle.

[4]

(c) Assembly language instructions are grouped.

Complete each statement by writing the name of the appropriate instruction group.

Loading data into the accumulator is an example of an instruction in the

..... group. Incrementing the index register is an example of an instruction in the group. Branching to another address is an example of an instruction in the group.

[3]

8. M/J 25/P11/Q6/a(ii,iii)

6 A programmer is buying a new computer.

(a) The programmer is considering the following two computers:

Computer 1	Quad-core 2.2 GHz processor 16-bit architecture 1 GB Random Access Memory (RAM) 500 GB magnetic hard disk
Computer 2	Dual-core 3.8 GHz processor 32-bit architecture 2 GB RAM 500 GB solid state drive (SSD)

(ii) The two computers have different amounts of RAM.

Explain how different amounts of RAM affect the performance of a computer.

[3]

(iii) The two computers have different bus widths.

Explain how different bus widths affect the performance of a computer.

[2]

9. M/J 25/P11/Q8/a,b

8 (a) The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
STO	<address>	Store the contents of ACC at the given address
ADD	#n/Bn/&n	Add the number n to the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
END		Return control to the operating system
ACC denotes Accumulator <address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current contents of memory are:

address Instruction

80	10
81	8
82	80
83	81
...	
200	LDD 81
201	INC ACC
202	STO 83
203	LDI 82
204	CMP 83
205	JPE 209
206	LDD 83
207	ADD #10
208	JMP 210
209	DEC ACC
210	STO 81
211	END

Trace the program currently in memory using the following trace table.

Instruction address	ACC	Memory address			
		80	81	82	83
		10	8	80	81

[4]

- (b) The table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

- (i) Write the bit manipulation instruction that can be used to set the least significant bit to 1 in an 8-bit register. All other bits must remain unchanged.

The instruction needs to work on a register that contains any 8-bit binary number.

[1]

- (ii) The ACC currently contains the following binary value.

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Write the result after the instruction `XOR &FE` is run.

--	--	--	--	--	--	--	--

[1]

- (iii) The ACC currently contains the following binary value.

0	1	1	0	1	0	1	1
---	---	---	---	---	---	---	---

Write the result after the instruction `LSR #5` is run.

--	--	--	--	--	--	--	--

[1]

10. M/J 25/P12/Q1/a,b,c

- 1 (a) The table has six statements about the Von Neumann model for a computer system. Three of the statements are incorrect.

Statement number	Statement
1	The Program Counter (PC) stores the next instruction to be fetched from memory.
2	The Arithmetic and Logic Unit (ALU) performs mathematical and logical operations.
3	The Control Unit (CU) sends signals to other components on the data bus.
4	The Memory Data Register (MDR) transfers data to the memory address stored in the Memory Address Register (MAR).
5	The MAR stores an address from memory.
6	The Accumulator (ACC) stores the result of calculations.

Complete the table by writing the **three** incorrect statement numbers and the corrected statements.

Incorrect statement number	Corrected statement

[3]

(b) Registers that are used in the Fetch-Execute (F-E) cycle include the PC, MAR, MDR and the ACC.

Identify **one** other register and describe its role in the Fetch-Execute (F-E) cycle.

Register

Role

[2]

(c) Explain how an interrupt from an input device will be detected and handled in the F-E cycle.

[4]

11. M/J 25/P13/Q2/a,b,c

2 A computer has a processor.

(a) The processor has a Control Unit (CU) and system clock.

(i) Explain how the CU and the system clock work together.

[2]

(ii) The processor runs the Fetch-Execute (F-E) cycle.

Write the stages of the F-E cycle using register transfer notation.

[4]

(b) The computer has cache memory.

Describe **one** benefit of the computer using cache memory.

[2]

(c) The computer connects to a monitor using a High Definition Multimedia Interface (HDMI) cable that connects into an HDMI port.

Explain how HDMI provides connection to peripheral devices.

[2]

12. M/J 25/P13/Q5/a,b

5 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
STO	<address>	Store the contents of ACC at the given address
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
END		Return control to the operating system

ACC denotes Accumulator
 <address> can be an absolute or a symbolic address
 # denotes a denary number, e.g. #123
 B denotes a binary number, e.g. B01001010
 & denotes a hexadecimal number, e.g. &4A

(a) The current contents of memory are:

Address	Instruction
50	47
51	48
52	49
53	50
...	
500	LDM #50
501	DEC ACC
502	CMP 51
503	JPE 505
504	JMP 501
505	STO 50
506	SUB #10
507	STO 51
508	END

- (i) Trace the program currently in memory using the following trace table.

Instruction address	ACC	Memory address			
		50	51	52	53
		47	48	49	50

[3]

- (ii) Complete the table by identifying **and** describing **two** modes of addressing that are **not** used in the program in part (a).

Mode of addressing	Description
.....	
.....	

[4]

- (b) The table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

Write the bit manipulation instructions that can be used to set only the most significant bit to 1 in an 8-bit register.

The instructions need to work on a register that contains any 8-bit binary number.

[2]

13. M/J 25/P13/Q7/a

7 (a) A computer can perform logical and arithmetic shifts.

- (i) Show the result of a logical left shift of 2 places on the two's complement binary integer 11001010

[1]

- (ii) Show the result of an arithmetic right shift of 3 places on the two's complement binary integer 10011110

[1]

14. O/N 24/P11/Q3/a,b

3 A computer system has a dual-core Central Processing Unit (CPU).

- (a) State the purpose of the system clock and the Control Unit (CU) in a CPU.

System clock

CU

[2]

- (b) (i) The number of cores in the processor affects the performance of the computer system.

Identify **one other** feature of a processor that can affect the performance of a computer system **and** state why it affects the performance.

Feature

Reason

[2]

- (ii) A solid state (flash) memory drive is automatically recognised by the computer when it is plugged into a port in the computer.

Identify an appropriate type of port to connect the solid state memory drive to the computer.

Explain how this port provides an automatic connection.

Port

Explanation

[3]

15. O/N 24/P11/Q8/a,b

- 8** A computer designed using the Von Neumann model for a computer system contains general purpose registers and special purpose registers.

(a) Describe the purpose of the Status Register (SR).

[2]

(b) Identify **two** differences between general purpose registers and special purpose registers.

1

2

[2]

16. O/N 24/P12/Q3/a,b

3 A student has a computer.

(a) The computer is designed using the Von Neumann model for a computer system.

Complete the table by describing the purpose of each of the given registers.

Register	Purpose
Program Counter (PC)	
Memory Address Register (MAR)	
Memory Data Register (MDR)	
Index Register (IX)	

[4]

(b) The student needs to connect the computer to a monitor that has a screen resolution of 2560×1600 pixels. The monitor also has built-in speakers.

The computer has a Video Graphics Array (VGA) port and a High Definition Multimedia Interface (HDMI) port.

Explain the benefits of connecting the monitor to the computer using the HDMI port instead of the VGA port.

[4]

17. O/N 24/P12/Q8/a,b

8 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
19	25
20	23
21	2
22	4
23	15
24	50
25	22

The current contents of the ACC and IX are shown:

ACC	50
IX	20

Complete the table by writing the content of the ACC and the IX after each set of instructions has run.

	Instructions	ACC content	IX content
1	LDM #19 DEC ACC		
2	LDD 23 ADD 19		
3	LDI 25 INC ACC		
4	LDR #21 LDX 2		

[5]

(b) The instruction set also includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end.
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current content of the ACC is shown:

ACC	1	0	0	1	1	0	1	0
------------	---	---	---	---	---	---	---	---

(i) The table has three sets of instructions. The binary number 10011010 is reloaded into the ACC before each set of instructions is run.

Complete the table by writing the content of the ACC after each set of instructions has run.

	Instructions	ACC content
1	LSL #2	
2	ADD #5 AND #30	
3	OR B11110010 INC ACC	

[3]

- (ii) Explain how bit manipulation can be used to test whether the binary number stored in the ACC represents an odd denary number.

Write the bit manipulation instruction that will be used.

Explanation

Instruction

[3]

18. O/N 24/P13/Q7/a,b

- 7 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
50	54
51	55
52	50
53	52
54	100
55	25
56	50

The current contents of the ACC and IX are shown:

ACC	50
IX	45

Complete the table by writing the content of the ACC after each program has run.

	Instructions	ACC content
1	LDD 50 ADD #4 ADD 54	
2	LDI 53 DEC ACC ADD 56	
3	LDM #55 SUB #5	

(b) The instruction set also includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

Explain how bit manipulation can be used to clear the data in an 8-bit register.

Write the bit manipulation instruction that will be used.

Explanation

Instruction

[3]

19. O/N 24/P13/Q8/c

(c) Describe the difference between a right logical binary shift and a right arithmetic binary shift.

[2]

20. M/J 24/P11/Q4/a,b

- 4 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
19	24
20	2
21	1
22	3
23	5
24	4
25	22

The current contents of the ACC and IX are shown:

ACC	12
IX	1

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDD 20 ADD #2	
2	LDX 22	
3	LDI 25 INC ACC SUB 22	
4	LDD 19 LDM #5 LDM #25	

(b) The processor includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current contents of memory are shown:

Address	Data
30	01110101
31	11111111
32	00000000
33	11001100
34	10101010

The current content of the ACC is shown:

1	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

Complete the table by writing the content of the ACC after each program has run.

The binary number 10011010 is reloaded into the ACC before each program is run.

Program number	Code	ACC content
1	AND 31	
2	XOR B01001111	
3	OR #30	

21. M/J 24/P12/Q5/a,b

- 5 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

- (a) The current contents of memory are shown:

Address	Data
10	1
11	3
12	5
13	11
14	10
15	16
16	12

The current contents of the ACC and IX are shown:

ACC	10
IX	0

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDI 15 SUB #1	
2	LDD 14 ADD 11	
3	LDM #11 ADD #3 SUB 16	
4	LDR #2 LDX 14 ADD #2	

(b) The processor includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current contents of memory are shown:

Address	Data
25	11000110
26	11100001
27	10000001
28	11001101
29	00001111

The current content of the ACC is shown:

0	1	0	0	0	1	1	0
---	---	---	---	---	---	---	---

Complete the table by writing the content of the ACC after each program has run.

The binary number 01000110 is reloaded into the ACC before each program is run.

Program number	Code	ACC content
1	XOR 29	
2	AND #29	
3	OR B11111111	

22. M/J 24/P13/Q3/a,b

- 3 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		



- (a) The current contents of memory are shown:

Address	Data
48	51
49	6
50	48
51	50
52	49
53	50
54	6

The current contents of the ACC and IX are shown:

ACC	2
IX	50

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDM #50 INC ACC SUB #1	
2	LDI 51 ADD 52	
3	LDR #2 LDX 50 DEC ACC	
4	LDD 52 SUB 54 INC ACC	

[4]



(b) The processor includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current contents of memory are shown:

Address	Data
100	00001101
101	10111110
102	11110011
103	00110111
104	00000000

The current content of the ACC is shown:

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---



Complete the table by writing the content of the ACC after each instruction has run.

The binary number 11111111 is reloaded into the ACC before each instruction is run.

Instruction number	Instruction	ACC content
1	LSL #2	
2	XOR 100	
3	AND 103	

[3]



23. O/N 23/P11/Q5/a,b,c

- 5 The Central Processing Unit (CPU) of the basic Von Neumann model for a computer system contains several special purpose registers.

- (a) The Memory Data Register (MDR), Index Register (IX) and the Accumulator (ACC) are examples of special purpose registers.

Identify **two other** special purpose registers **and** state their role in the CPU.

Special purpose register 1

Role

Special purpose register 2

Role

[4]

- (b) Describe what is meant by the **Immediate Access Store (IAS)** in a computer system.

[2]

- (c) A computer has a single 2.1 GHz CPU.

- (i) Describe how increasing the clock speed to 4 GHz can increase the performance of the computer.

[1]

- (ii) A second computer has a CPU with two 2.1 GHz cores.

Explain why the second computer does not always run twice as fast as the computer with one 2.1 GHz CPU.

[5]

24. O/N 23/P11/Q8/a,b,c

- 8 (a) Identify the purpose of the first pass of a two-pass assembler.

[1]

- (b) The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
OUT		Output to the screen the character whose ASCII value is stored in ACC
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123		

- (i) Give **one** example of an instruction that belongs to **each** of the following instruction groups.

Only use the instructions given in the table. Each instruction must have a suitable operand.

Data movement

Arithmetic operation

Conditional instruction

[3]

- (ii) The instruction `LDR #2` uses immediate addressing.

Give **one** similarity and **one** difference between direct addressing and indexed addressing.

Similarity

Difference

[2]

- (iii) Identify **one other** mode of addressing.

[1]

(c) The following table shows another part of the instruction set for the same processor.

Instruction		Explanation
Opcode	Operand	
AND	Bn	Bitwise AND operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the operand
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
# denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(i) The current contents of the ACC are:

0	1	0	0	1	1	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

AND B10100101

--	--	--	--	--	--	--	--

[1]

(ii) The current contents of the ACC are:

0	0	0	1	0	1	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

LSR #3

--	--	--	--	--	--	--	--

[1]

(iii) The current contents of the ACC are:

1	1	1	1	0	1	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

XOR B00100101

--	--	--	--	--	--	--	--

[1]

25. O/N 23/P12/Q8/c

8 A computer has an Operating System (OS).

(c) An optical disc reader/writer is connected to the computer.

(i) Give the name of **one** port that can provide a connection for the optical disc reader/writer.

[1]

(ii) Describe the roles of the address bus, the data bus **and** buffers in the process of writing data to the optical disc reader/writer.

[3]

26. O/N 23/P12/Q9/a,b

9 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store the contents of ACC at the given address
ADD	#n	Add the denary number n to the ACC
JMP	<address>	Jump to the given address
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123		

(a) The instructions in the processor's instruction set can be grouped according to their function.

Identify the instruction group for each of the following opcodes.

IN

ADD

JPE

CMI

[4]

- (b)** The current contents of main memory and selected values from the ASCII character set are given on page 15.

Trace the program currently in memory using the trace table when the input is '1'.

Address	Instruction	Instruction address	ACC	IX	Memory address				Output
					100	101	110	111	
10	LDR #0				0	0	51	65	
11	IN								
12	STO 101								
13	LDX 110								
14	CMP 100								
15	JPE 21								
16	LDD 101								
17	ADD #16								
18	INC IX								
19	STO 100								
20	JMP 13								
21	OUT								
22	END								
...	↗								
100	0								
101	0								
...	↗								
110	51								
111	65								

ASCII value	Character
49	1
50	2
51	3
52	4
...	↗
65	A
66	B
67	C
68	D

27. O/N 23/P13/Q7/d

- 7** A laptop computer has Static RAM (SRAM).

A virtual reality headset and a laser printer are connected to the laptop.

- (d)** Identify **one** port that could be used to connect the virtual reality headset to the laptop.

Justify your choice.

Port

Justification

[3]

28. O/N 23/P13/Q9/a,b,c,d

- 9** A computer system is designed using the basic Von Neumann model.

Registers and buses are components in the Von Neumann model.

- (a) (i)** Identify **three other** components in the Von Neumann model of a computer system.

Do not include registers or buses in your answers.

1

2

3

[3]

- (ii)** Identify **two** differences between special purpose registers and general purpose registers.

1

2

[2]

- (b)** The following incomplete table contains steps of the Fetch-Execute (F-E) cycle and their descriptions.

Complete the table by writing the missing steps using register transfer notation **and** the missing descriptions.

Step	Description
.....	The address in PC is incremented.
$MDR \leftarrow [[MAR]]$	
$MAR \leftarrow [PC]$	
.....	The contents of MDR are copied into CIR.

[4]

- (c) Interrupts can be caused by software programs or hardware devices.

State **one** cause of a software interrupt.

[1]

- (d) The following statements describe the stages that the CPU performs when an interrupt is detected.

There are **three** missing statements.

Write the letter of the missing statements from the table in the correct place to complete the description.

- At the end of each Fetch-Execute (F-E) cycle, the processor checks if an interrupt flag is set.
-
- If the interrupt priority is high enough, the processor saves the current contents of the registers.
-
- When servicing of the interrupt is complete, the processor restores the registers.

6

Letter	Stage
A	The address of the Interrupt Service (ISR) handling routine is loaded into the Program Counter (PC).
B	Lower priority interrupts are re-enabled.
C	The device causing the interrupt transfers data to the CPU.
D	The processor identifies the source of the interrupt and checks the priority of the interrupt.
E	The ISR is incremented.

[3]

29. M/J 23/P11/Q3/d(vi)

3 A computer has an Operating System (OS).

(d) The computer stores data in binary form.

(vi) Show the result of a 3-place right logical shift on the binary number:

11001100

[1]

30. M/J 23/P11/Q6

6 An interrupt is generated when a key is pressed on a computer keyboard.

Explain how the computer handles this interrupt.

[5]

31. M/J 23/P12/Q3/a,b

3 A program is written in assembly language.

(a) The program is converted into machine code by a two-pass assembler.

Draw **one or more** lines to identify the pass or passes in which each action takes place.

Action	Pass
generates object code	
reads the source code one line at a time	first
removes white space	second
adds labels to the symbol table	

[3]

(b) Assembly language statements can use different modes of addressing.

Complete the following description of addressing modes.

..... addressing is when the operand holds the memory address of the data.

..... addressing is when the operand holds a memory address that stores the memory address of the data.

..... addressing is when the operand is the data.

[3]

32. M/J 23/P12/Q4/d

4 Data in a computer is stored in binary form.

(d) Show the result of a 2-place left logical shift on the binary number:

01001111

[1]

33. M/J 23/P12/Q5/a,b,c

5 A student has purchased a new laptop.

(a) The laptop is designed using the Von Neumann model for a computer system.

Identify **two** types of signal that a control bus can transfer.

1

2

[2]

(b) Describe **two** ways the hardware of a laptop can be upgraded to improve the performance **and** explain how each upgrade improves the performance.

1

2

[4]

(c) Peripherals are connected to the laptop using ports.

(i) A printer is connected to a Universal Serial Bus (USB) port.

Describe how data is transmitted through a USB port.

[1]

(ii) A monitor is connected to the laptop using a different type of port.

Identify **one** other type of port that can be used to connect the monitor.

[1]

34. M/J 23/P13/Q7/b,c

7 A computer stores data in binary form.

(b) The computer has a Control Unit (CU), system clock and control bus.

Explain how the CU, system clock and control bus operate to transfer data between the components of the computer system.

[4]

(c) Complete the table by writing the register transfer notation for each stage of the Fetch-Execute (F-E) cycle given in the table.

Stage description	Register transfer notation
The Program Counter (PC) is incremented	
The data in the address stored in the Memory Address Register (MAR) is copied to the Memory Data Register (MDR)	

[2]

35. O/N 22/P11/Q5/a,b,c,d

- 5 (a) State what is meant by the **stored program concept** in the Von Neumann model of a computer system.

[1]

- (b) A Central Processing Unit (CPU) contains several special purpose registers and other components.

- (i) State the role of the following registers.

Program Counter (PC)

Index Register (IX)

Status Register (SR)

[3]

- (ii) Tick (✓) **one** box in each row to identify the system bus used by each CPU component.

CPU component	Data bus	Address bus	Control bus
System clock			
Memory Address Register (MAR)			

[1]

- (iii) Describe the purpose of the Control Unit (CU) in a CPU.

[2]

- (c) Describe the purpose of an interrupt in a computer system.

[2]

- (d) Identify **two** causes of a software interrupt.

1

2

[2]

36. O/N 22/P11/Q6/a,b,c

- 6 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

- (a) The current contents of main memory and selected values from the ASCII character set are given.

(ii) The following instructions are repeated for your reference.

Instruction		Explanation
Opcode	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store contents of ACC at the given address

State the purpose of this part of an assembly language program.

```
LDD 100
STO 165
LDD 101
STO 100
LDD 165
STO 101
```

[1]

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	Bn	Bitwise AND operation of the contents of ACC with the binary number n
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the binary number n
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	Bn	Bitwise OR operation of the contents of ACC with the binary number n
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(i) The current contents of the ACC are:

1	0	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the result after the execution of the following instruction.

```
XOR B00011111
```

--	--	--	--	--	--	--	--

[1]

(ii) The current contents of the ACC are:

1	0	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the result after the execution of the following instruction.

AND B11110000

--	--	--	--	--	--	--	--

[1]

(iii) The current contents of the ACC are:

1	0	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the result after the execution of the following instruction.

OR B11001100

--	--	--	--	--	--	--	--

[1]

(iv) The current contents of the ACC are:

1	0	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the result after the execution of the following instruction.

LSR #2

--	--	--	--	--	--	--	--

[1]

(c) Tick (✓) **one or more** boxes in each row to indicate whether the task is performed in the first pass or the second pass of a two-pass assembler.

Task	First pass	Second pass
Remove comments.		
Read the assembly language program one line at a time.		
Generate the object code.		
Check the opcode is in the instruction set.		

[2]

37. O/N 22/P11/Q7/d,e

- (d) Explain the reasons why increasing the amount of cache memory can improve the performance of a CPU.

[2]

- (e) State the name of a peripheral device port that provides a physical connection in the computer for each of these peripherals.

3D printer

Monitor

[2]

38. O/N 22/P12/Q7/a,b,c

- 7 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store the contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
END		Return control to the operating system
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

- (a) Trace the program currently in memory using the trace table, stopping when line 90 is executed for a second time.

[5]

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the binary number n
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

The contents of memory addresses 50 and 51 are shown:

Memory address	Data value
50	01001101
51	10001111

(i) The current contents of the ACC are:

0	1	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

XOR B00011111

--	--	--	--	--	--	--	--

[1]

(ii) The current contents of the ACC are:

0	1	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

AND 50

--	--	--	--	--	--	--	--

[1]

(iii) The current contents of the ACC are:

0	1	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

LSL #3

--	--	--	--	--	--	--	--

[1]

(iv) The current contents of the ACC are:

0	1	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

OR 51

--	--	--	--	--	--	--	--

[1]

(c) Write the register transfer notation for each of the stages in the fetch-execute cycle described in the table.

Description	Register transfer notation
Copy the address of the next instruction into the Memory Address Register.	
Increment the Program Counter.	
Copy the contents of the Memory Data Register into the Current Instruction Register.	

[3]

39. O/N 22/P13/Q4/a,b,c

- 4 (a)** A Central Processing Unit (CPU) contains several special purpose registers and other components.

(i) State the roles of the following registers.

Memory Address Register (MAR)

Memory Data Register (MDR)

[2]

(ii) State when interrupts are detected during the Fetch-Execute (F-E) cycle.

[1]

- (b)** A computer system contains a system clock.

Describe the purpose of the system clock.

[2]

- (c)** Upgrading secondary storage to solid state typically improves the performance of computer systems.

Identify **one other** upgrade to the hardware **and** explain why it improves the performance of a computer system.

Upgrade

Explanation

[2]

40. O/N 22/P13/Q6/a,b,c

- 6** The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
MOV	<register>	Moves the contents of the accumulator to the given register (IX)
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
SUB	#n	Subtract the denary number n from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	#n	Compare the contents of ACC with number n
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

- (a) The current contents of main memory and selected values from the ASCII character set are given.

(i) Trace the program currently in memory using the trace table.

Address	Instruction	Instruction address	ACC	IX	Memory address					Output
					100	101	110	111	112	
75	LDR #0				1	0	97	98	97	
76	LDD 100									
77	CMP #2									
78	JPE 91									
79	LDX 110									
80	SUB #32									
81	CMP #65									
82	JPN 86									
83	LDM #1									
84	ADD 101									
85	STO 101									
86	LDM #1									
87	ADD 100									
88	STO 100									
89	INC IX									
90	JMP 76									
91	LDD 101									
92	ADD #48									
93	OUT									
94	END									
...										
100	1									
101	0									
...										
110	97									
111	98									
112	97									

ASCII value	Character
49	1
50	2
51	3
52	4
65	A
66	B
67	C
68	D

[4]

(ii) Explain the purpose of **relative addressing** in an assembly language program.

[2]

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	Bn	Bitwise AND operation of the contents of ACC with the binary number n
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	Bn	Bitwise OR operation of the contents of ACC with the binary number n
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(i) The current contents of the ACC are:

0	0	1	1	0	1	1	0
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

AND B01001100

--	--	--	--	--	--	--	--

[1]

(ii) The current contents of the ACC are:

1	0	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

OR B01001111

--	--	--	--	--	--	--	--

[1]

(iii) The current contents of the ACC are:

1	0	0	1	1	1	0	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

LSR #2

--	--	--	--	--	--	--	--

[1]

(c) One instruction group is data movement.

Give the name of **one other** instruction group.

[1]

41. O/N 21/P11/Q6/a,b

6 (a) There are **two** errors in the following register transfer notation for the fetch-execute cycle.

- 1 MAR $\leftarrow$ [PC]
- 2 PC $\leftarrow$ [PC] - 1
- 3 MDR $\leftarrow$ [MAR]
- 4 CIR $\leftarrow$ [MDR]

Complete the following table by:

- identifying the line number of each error
- describing the error
- writing the correct statement.

Line number	Description of the error	Correct statement

[4]

(b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store the contents of ACC at the given address
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

The current contents of main memory are shown:

Address	Data
100	00001111
101	11110000
102	01010101
103	11111111
104	00000000

Each row of the following table shows the current contents of ACC in binary and the instruction that will be performed on those contents.

Complete the table by writing the new contents of the ACC after the execution of each instruction.

Current contents of the ACC	Instruction	New contents of the ACC
11111111	OR 101	
00000000	XOR #15	
10101010	LSR #2	
01010101	AND 104	

[4]

42. O/N 21/P12/Q7/a

7 A computer has system software.

(a) The Operating System handles interrupts.

Tick (✓) **one** box in each row to identify whether each event is an example of a hardware interrupt or a software interrupt.

Event	Hardware interrupt	Software interrupt
Buffer full		
Printer is out of paper		
User has pressed a key on the keyboard		
Division by zero		
Power failure		
Stack overflow		

[3]

43. O/N 21/P12/Q8/a,b

8 The Von Neumann model for a computer system uses registers.

(a) Describe the role of the following special purpose registers in the fetch-execute (F-E) cycle.

(i) Memory Address Register (MAR)

Memory Data Register (MDR)

[4]

(ii) Another special purpose register is the Index Register.

Identify **one other** special purpose register used in the Von Neumann model for a computer system.

[1]

- (b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store the contents of ACC at the given address
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123		

The current contents of main memory are shown:

Address	Data
100	01010101
101	11110000
102	00001111
103	00000000
104	11111111

- (i) In the following table, each row shows the current contents of the ACC in binary and the instruction that will be performed on those contents.

Complete the table by writing the new contents of the ACC after the execution of each instruction.

Current contents of the ACC	Instruction	New contents of the ACC
01010101	XOR 101	
11110000	AND 104	
00001111	LSL #4	
11111111	OR 102	

[4]

- (ii) The following table contains five assembly language instruction groups.

Write an appropriate assembly language instruction for each instruction group, using the given instruction set. The first one has been completed for you.

Instruction Group	Instruction
Data movement	LDM #2
Input and output of data	
Arithmetic operations	
Unconditional and conditional instructions	
Compare instructions	

[4]

- (iii) The opcode **LDM** uses immediate addressing. The opcode **LDD** uses direct addressing.

Identify **and** describe **one additional** mode of addressing.

Mode of addressing

Description

[2]

44. O/N 21/P13/Q6/a,b

- 6 (a) There are **two** errors in the following register transfer notation for the fetch-execute cycle.

```
1  MAR ← [PC]

2  PC ← [PC] - 1

3  MDR ← [MAR]

4  CIR ← [MDR]
```

- Complete the following table by:
- identifying the line number of each error
 - describing the error
 - writing the correct statement.

Line number	Description of the error	Correct statement

[4]

(b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store the contents of ACC at the given address
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

The current contents of main memory are shown:

Address	Data
100	00001111
101	11110000
102	01010101
103	11111111
104	00000000

Each row of the following table shows the current contents of ACC in binary and the instruction that will be performed on those contents.

Complete the table by writing the new contents of the ACC after the execution of each instruction.

Current contents of the ACC	Instruction	New contents of the ACC
11111111	OR 101	
00000000	XOR #15	
10101010	LSR #2	
01010101	AND 104	

[4]

45. M/J 21/P11/Q3/a,b,c

- 3** A processor has one general purpose register, the Accumulator (ACC), and several special purpose registers.

- (a)** Complete the following description of the role of the registers in the fetch-execute cycle by writing the missing registers.

The holds the address of the next instruction to be loaded. This address is sent to the

The holds the data fetched from this address.

This data is sent to the and the Control Unit decodes the instruction's opcode.

The is incremented.

[5]

- (b)** The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing: The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
MOV	<register>	Move the contents of the accumulator to the given register (IX)
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
<address> can be an absolute address or a symbolic address # denotes a denary number, e.g. #123		

The current contents of the main memory and selected values from the ASCII character set are shown.

Address	Instruction
200	LDD 365
201	CMP 366
202	JPE 209
203	INC ACC
204	STO 365
205	MOV IX
206	LDX 365
207	OUT
208	JMP 200
209	END
...	
365	1
366	3
367	65
368	66
IX	0

ASCII code table (selected codes only)	
ASCII code	Character
65	A
66	B
67	C
68	D

Write the contents of the Accumulator after the processor has executed the following instruction:

LSL #2

--	--	--	--	--	--	--	--

[1]

(ii) The Accumulator currently contains the binary number:

0	0	1	1	0	1	0	1
---	---	---	---	---	---	---	---

Identify the mathematical operation that the following instruction will perform on the contents of the accumulator.

LSR #3

[1]

46. M/J 21/P12/Q4/a,b(ii,iii)

- 4 The table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
# denotes a denary number, e.g. #123		

The current contents of the main memory and selected values from the ASCII character set are:

Address Instruction

70	IN
71	CMP 100
72	JPE 80
73	CMP 101
74	JPE 76
75	JMP 80
76	LDD 102
77	INC ACC
78	STO 102
79	JMP 70
80	LDD 102
81	DEC ACC
82	STO 102
83	JMP 70
...	
100	68
101	65
102	100

ASCII code table (selected codes only)

ASCII code	Character
65	A
66	B
67	C
68	D

- (a) Complete the trace table for the program currently in main memory when the following characters are input:

A D

(b) Some bit manipulation instructions are shown in the table:

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute address or a symbolic address # denotes a denary number, e.g. #123		

The contents of the memory address 300 are shown:

Bit Number	7	6	5	4	3	2	1	0
300	0	1	1	0	0	1	1	0

- (ii) An assembly language program needs to test if bit number 2 in memory address 300 is a 1.

Complete the assembly language instruction to perform this test.

#4

[1]

- (iii) An assembly language program needs to set bit numbers 4, 5, 6 and 7 to 0, but keep bits 0 to 3 with their existing values.

Write the assembly language instruction to perform this action.

[2]

47. M/J 21/P12/Q5/a,b

5 Seth uses a computer for work.

- (a) Complete the following descriptions of internal components of a computer by writing the missing terms.

The transmits the signals to coordinate events based on the electronic pulses of the

The carries data to the components, while the carries the address where data needs to be written to or read from.

The performs mathematical operations and logical comparisons.

[5]

- (b) Describe the ways in which the following factors can affect the performance of his laptop computer.

Number of cores

Clock speed

[4]

48. M/J 21/P13/Q3/a,b,c

3 A processor has one general purpose register, the Accumulator (ACC), and several special purpose registers.

- (a) Complete the following description of the role of the registers in the fetch-execute cycle by writing the missing registers.

The holds the address of the next instruction to be loaded. This address is sent to the

The holds the data fetched from this address.

This data is sent to the and the Control Unit decodes the instruction's opcode.

The is incremented.

[5]

- (b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing: The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
MOV	<register>	Move the contents of the accumulator to the given register (IX)
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
<address> can be an absolute address or a symbolic address # denotes a denary number, e.g. #123		

The current contents of the main memory and selected values from the ASCII character set are shown.

Address	Instruction
200	LDD 365
201	CMP 366
202	JPE 209
203	INC ACC
204	STO 365
205	MOV IX
206	LDX 365
207	OUT
208	JMP 200
209	END
...	
365	1
366	3
367	65
368	66
IX	0

ASCII code table (selected codes only)	
ASCII code	Character
65	A
66	B
67	C
68	D

Write the contents of the Accumulator after the processor has executed the following instruction:

LSL #2

--	--	--	--	--	--	--	--

[1]

(ii) The Accumulator currently contains the binary number:

0	0	1	1	0	1	0	1
---	---	---	---	---	---	---	---

Identify the mathematical operation that the following instruction will perform on the contents of the accumulator.

LSR #3

[1]

Answers Section

1. M/J 26/P11/Q3/a,b,c

3(a)	1 mark for each correct row <table border="1" data-bbox="236 349 1018 674"> <thead> <tr> <th>accumulator current data</th><th>set of instructions</th><th>new accumulator data</th></tr> </thead> <tbody> <tr> <td>B11001010</td><td>ADD B00000010 INC ACC</td><td>B11001101</td></tr> <tr> <td>#100</td><td>LDM #200 ADD 200</td><td>#301</td></tr> <tr> <td>#103</td><td>LDD 103 DEC ACC DEC ACC</td><td>#48</td></tr> </tbody> </table>	accumulator current data	set of instructions	new accumulator data	B11001010	ADD B00000010 INC ACC	B11001101	#100	LDM #200 ADD 200	#301	#103	LDD 103 DEC ACC DEC ACC	#48	3 Correct answers only
accumulator current data	set of instructions	new accumulator data												
B11001010	ADD B00000010 INC ACC	B11001101												
#100	LDM #200 ADD 200	#301												
#103	LDD 103 DEC ACC DEC ACC	#48												
3(b)(i)	1 mark for AND 1 mark for #0 // &0 // B00000000 Answer: AND B00000000 // AND &0 // AND #0	2 Accept correct shift instructions Bit manipulation so no load commands												
3(b)(ii)	1 mark for the correct answer 0000 0110	1 Correct answer only												
3(c)	1 mark for a correct name 1 mark for a matching description 1 mark for a reasonable example - Indirect - the operand stores the address where the data is found // the data is held in the address stored in the given address - example: LDI 23 loads the data in the address stored in address 23 - Indexed - the address is calculated by adding the operand to the contents of the index register - example: LDX 20 adds 20 to the contents of the index register to calculate the address	3 Do not accept Immediate, relative, direct, absolute, symbolic.												

2. M/J 26/P13/Q7/a,b,c

7(a)	1 mark for each correct answer	4	No mnemonics / opcodes										
	<table><tr><td>Mode of addressing</td><td>Description</td></tr><tr><td>Direct</td><td>The operand is an address where the data is stored.</td></tr><tr><td>Indirect</td><td>The operand is an address that stores the address where the data is stored.</td></tr><tr><td>Indexed</td><td>The operand is an address. The address where the data is stored is calculated by adding the operand to the content of the index register.</td></tr><tr><td>Immediate</td><td>The operand is the data to be used.</td></tr></table>			Mode of addressing	Description	Direct	The operand is an address where the data is stored.	Indirect	The operand is an address that stores the address where the data is stored.	Indexed	The operand is an address. The address where the data is stored is calculated by adding the operand to the content of the index register.	Immediate	The operand is the data to be used.
	Mode of addressing			Description									
	Direct			The operand is an address where the data is stored.									
	Indirect			The operand is an address that stores the address where the data is stored.									
	Indexed			The operand is an address. The address where the data is stored is calculated by adding the operand to the content of the index register.									
Immediate	The operand is the data to be used.												

7(b)	1 mark each to max 3 - Bit manipulation allows a change/evaluation of the state/value of a bit // masking Monitoring: - When monitoring, bit manipulation can be used to establish the state of a device (read inputs) ... by testing/observing the state of an individual bit Controlling: - When controlling bit manipulation can be used to change the state of a device (control output) ... by setting/clearing/toggling a single bit ... or by clearing a byte	3	
7(c)	1 mark for similarity: - Both lose the Least Significant Bit(s) (LSB) // both lose bit(s) from the right of the number 1 mark for difference: - Arithmetic retains sign/MSB while logical does not retain sign/MSB - MSBs are filled with 0 for logical and copies of the sign bit for arithmetic.	2	For difference, must have both sides

3. O/N 25/P11/Q4/a,b,c,d

4(a)	1 mark for: LSL #2	1								
4(b)	<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	1	1	1	1	0	0	0	1	1
1	1	1	1	0	0	0	1			
4(c)	<table><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr></table>	0	0	1	0	0	0	1	1	1
0	0	1	0	0	0	1	1			
4(d)	<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td></tr></table>	1	1	1	1	0	0	1	1	1
1	1	1	1	0	0	1	1			

4. O/N 25/P11/Q6/a,b

6(a)	1 mark for each correct description, max 2 marks		2
	Register	Role	
	ACC	stores the intermediate results of arithmetic and logical operations // holds the result of a calculation	
	CIR	holds the instruction currently being decoded and/or executed	

6(b)	1 mark per bullet point, max 2 marks e.g. • Latency may be increased • ... because the cores must communicate with one another • There is a potential for dead-lock situations • ... where one core may wait for information from other cores which in turn are waiting for the first one • Not all software is designed to use multi-cores • ... so some of the additional cores would be idle • Increased heat generation • ... which could cause damage to other components	2
------	--	----------

5. O/N 25/P12/Q3/a,b,c

3(a)(i)	1 mark The value of the operand is an offset value which is added to another base value to give the address from which the contents are loaded to the accumulator	1								
3(a)(ii)	1 mark per bullet point, max 2 marks - Program counter (PC) - Memory Data Register (MDR) - Memory Address Register (MAR) - Current Instruction Register (CIR) - Status Register	2								
3(b)	1 mark for each correct value, max 3 marks <table><tr><th>Instruction</th><th>Value in ACC</th></tr><tr><td>LDM #98</td><td>98</td></tr><tr><td>LDI 101</td><td>8</td></tr><tr><td>LDX 100</td><td>32</td></tr></table>	Instruction	Value in ACC	LDM #98	98	LDI 101	8	LDX 100	32	3
Instruction	Value in ACC									
LDM #98	98									
LDI 101	8									
LDX 100	32									
3(c)	1 mark per bullet point, max 2 marks for each Clock Speed: - Processor can perform more F-E cycles per second ... so more instructions / data can be processed each second Cache Memory: - Can store more of the most frequently used instructions ... which reduces the need to access slower RAM	4								

6. O/N 25/P13/Q2/c

2(c)	1 mark 1111 0010	1
------	--------------------------------	----------

7. O/N 25/P13/Q6/a,b,c

6(a)	1 mark for each correct row <table border="1"> <thead> <tr> <th>Current contents of the ACC</th><th>Instruction</th><th>Contents of the ACC after the execution of the instruction</th></tr> </thead> <tbody> <tr> <td>0000 1111</td><td>AND 101</td><td>0000 1110</td></tr> <tr> <td>0000 0000</td><td>LDM #100</td><td>0110 0100</td></tr> <tr> <td>0000 0001</td><td>XOR &F1</td><td>1111 0000</td></tr> <tr> <td>0001 0001</td><td>CMP 101</td><td>0001 0001</td></tr> </tbody> </table>	Current contents of the ACC	Instruction	Contents of the ACC after the execution of the instruction	0000 1111	AND 101	0000 1110	0000 0000	LDM #100	0110 0100	0000 0001	XOR &F1	1111 0000	0001 0001	CMP 101	0001 0001	4
Current contents of the ACC	Instruction	Contents of the ACC after the execution of the instruction															
0000 1111	AND 101	0000 1110															
0000 0000	LDM #100	0110 0100															
0000 0001	XOR &F1	1111 0000															
0001 0001	CMP 101	0001 0001															
6(b)	1 mark for each bullet point, max 4 marks - MAR stores the <u>address</u> of the next instruction / data to be read from / or written to memory - The address is received from the Program Counter (PC) - The MDR stores the data / instruction in the address stored in the MAR which has been read / written - The instruction passes to the CIR for decoding and executing	4															
6(c)	1 mark for each bullet point, max 3 marks - Data movement - Arithmetic operations - Conditional and unconditional (jump) instructions	3															

8. M/J 25/P11/Q6/a(ii,iii)

6(a)(ii)	1 mark each to max 3 - More RAM means more currently running data and instructions can be stored - without needing to use virtual memory - without having to fetch the data from secondary storage first - which has a slower access time - Less latency / delay waiting for instructions / data	3
----------	--	----------

6(a)(iii)	1 mark each to max 2 e.g. - Wider data bus means more data can be transferred between components at a time - there is less delay / latency when fetching data for a running process - Wider address bus means larger memory addresses can be used - allowing more memory locations to be accessed directly - so less likely to run out of memory	2
-----------	---	----------

9. M/J 25/P11/Q8/a,b

8(a)	1 mark for each shaded section	4																																																																						
	<table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th colspan="4">Memory address</th></tr><tr><th>80</th><th>81</th><th>82</th><th>83</th></tr><tr><td></td><td></td><td>10</td><td>8</td><td>80</td><td>81</td></tr><tr><td>200</td><td>8</td><td></td><td></td><td></td><td></td></tr><tr><td>201</td><td>9</td><td></td><td></td><td></td><td></td></tr><tr><td>202</td><td></td><td></td><td></td><td></td><td>9</td></tr><tr><td>203</td><td>10</td><td></td><td></td><td></td><td></td></tr><tr><td>204</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>206</td><td>9</td><td></td><td></td><td></td><td></td></tr><tr><td>207</td><td>19</td><td></td><td></td><td></td><td></td></tr><tr><td>208</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>210</td><td></td><td></td><td>19</td><td></td><td></td></tr></table>	Instruction address	ACC	Memory address				80	81	82	83			10	8	80	81	200	8					201	9					202					9	203	10					204						206	9					207	19					208						210			19			
Instruction address	ACC			Memory address																																																																				
		80	81	82	83																																																																			
		10	8	80	81																																																																			
200	8																																																																							
201	9																																																																							
202					9																																																																			
203	10																																																																							
204																																																																								
206	9																																																																							
207	19																																																																							
208																																																																								
210			19																																																																					
8(b)(i)	1 mark for: OR B0000 0001 // OR #1 // OR &1	1																																																																						
8(b)(ii)	1 mark for: 1010 1011	1																																																																						
8(b)(iii)	1 mark for: 0000 0011	1																																																																						

10. M/J 25/P12/Q1/a,b,c

1(a)	1 mark for each correct row <table><tr><th>Incorrect statement number</th><th>Corrected statement</th></tr><tr><td>1</td><td>The Program Counter (PC) stores the address of the next instruction to be fetched from memory.</td></tr><tr><td>3</td><td>The Control Unit (CU) sends signals to other components on the control bus.</td></tr><tr><td>4</td><td>The Memory Data Register (MDR) holds data to be stored in the memory address in the MAR. // The Memory Data Register (MDR) holds data read from the memory address in the MAR</td></tr></table>	Incorrect statement number	Corrected statement	1	The Program Counter (PC) stores the address of the next instruction to be fetched from memory.	3	The Control Unit (CU) sends signals to other components on the control bus.	4	The Memory Data Register (MDR) holds data to be stored in the memory address in the MAR. // The Memory Data Register (MDR) holds data read from the memory address in the MAR	3
Incorrect statement number	Corrected statement									
1	The Program Counter (PC) stores the address of the next instruction to be fetched from memory.									
3	The Control Unit (CU) sends signals to other components on the control bus.									
4	The Memory Data Register (MDR) holds data to be stored in the memory address in the MAR. // The Memory Data Register (MDR) holds data read from the memory address in the MAR									
1(b)	1 mark for name, 1 mark for corresponding role e.g. • Current Instruction Register // CIR • To store the instruction to be decoded / executed next • Status register // SR • To contain bits that can be referenced individually to indicate a state or event • Interrupt register • To store details of any interrupts that have occurred	2								
1(c)	1 mark each to max 4 • At the start / end of FE cycle the interrupt register is checked • The priority of any interrupts waiting is checked • if the priority of the interrupt is higher than the current process • the contents of the registers are stored on the stack • The relevant Interrupt Service Routine (ISR) / interrupt handler is called to process the interrupt • When the ISR has finished, a further check is made for higher priority interrupts • if no more interrupts of higher priority, the register contents are restored and the next FE cycle continues	4								

11. M/J 25/P13/Q2/a,b,c

2(a)(i)	1 mark each to max 2 - max 1 for each set of marks • The control unit synchronises the actions of the processor • by sending a command / signal on each timing signal produced by the system clock • using / along the control bus	2
---------	---	----------

2(a)(ii)	1 mark each • $MAR \leftarrow [PC]$ and $PC \leftarrow [PC] + 1$ • $MDR \leftarrow [[MAR]]$ • $CIR \leftarrow [MDR]$ • Correct order • For example: • $MAR \leftarrow [PC]$ • $PC \leftarrow [PC] + 1$ • $MDR \leftarrow [[MAR]]$ • $CIR \leftarrow [MDR]$	4
2(b)	1 mark each • Using cache memory improves system performance • because cache is fast access memory close to the CPU • which stores frequently used instructions / data • so that they can be accessed faster than from RAM	2
2(c)	1 mark each to max 2 e.g. • HDMI transfers both audio and video using a single cable • HDMI has a high bandwidth • Data is transmitted in a stream • of uncompressed digital signals • HDMI uses a technology called Transition-Minimized Differential Signalling (TMDS)	2

12. M/J 25/P13/Q5/a,b

5(a)(i)	1 mark for each shaded section <table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th colspan="4">Memory address</th></tr><tr><th>50</th><th>51</th><th>52</th><th>53</th></tr><tr><td></td><td></td><td>47</td><td>48</td><td>49</td><td>50</td></tr><tr><td>500</td><td>50</td><td></td><td></td><td></td><td></td></tr><tr><td>501</td><td>49</td><td></td><td></td><td></td><td></td></tr><tr><td>502</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>504</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>501</td><td>48</td><td></td><td></td><td></td><td></td></tr><tr><td>502</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>503</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>505</td><td></td><td>48</td><td></td><td></td><td></td></tr><tr><td>506</td><td>38</td><td></td><td></td><td></td><td></td></tr><tr><td>507</td><td></td><td></td><td>38</td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Instruction address	ACC	Memory address				50	51	52	53			47	48	49	50	500	50					501	49					502						504						501	48					502						503						505		48				506	38					507			38															3
Instruction address	ACC			Memory address																																																																																						
		50	51	52	53																																																																																					
		47	48	49	50																																																																																					
500	50																																																																																									
501	49																																																																																									
502																																																																																										
504																																																																																										
501	48																																																																																									
502																																																																																										
503																																																																																										
505		48																																																																																								
506	38																																																																																									
507			38																																																																																							
5(a)(ii)	1 mark for addressing mode, 1 mark for matching description • Direct • The operand is the address of the data • Indirect • The operand points to the memory location which is the address of the data • Indexed • The address of the data is formed by adding the contents of the Index Register (IX) to the operand	4																																																																																								
5(b)	1 mark for each correct instruction e.g. AND B00000000 / #0 / &0 OR B10000000 / #128 / &80	2																																																																																								

13. M/J 25/P13/Q7/a

7(a)(i)	1 mark for 0010 1000	1
7(a)(ii)	1 mark for 1111 0011	1

14. O/N 24/P11/Q3/a,b

3(a)	1 mark for system clock and 1 mark for Control Unit System clock • To synchronise operations • ... by creating and transmitting timing signals on the control bus Control Unit • Sends/receives control signals along control bus • Reads an instruction from the contents of the memory location whose address is stored in PC • Coordinates/synchronises the activity of other components in the CPU • Manages the execution of instructions • Controls communication between the components in the CPU	2
3(b)(i)	1 mark for the feature, 1 mark max for the matching reason e.g. Feature: clock speed • Higher clock speed means that more F-E cycles are executed per second // Higher clock speed results in more throughput Feature: bus width • Larger bus width means that more data transferred at the same time	2
3(b)(ii)	1 mark for correct port. 2 marks for explanation Port: USB / Universal Serial Bus Explanation: • A voltage change occurs when the drive is plugged in • The computer detects this voltage change • The code of the device is transferred to computer • ... the OS finds the code of the device in the list of devices • ... and loads the appropriate device driver	3

15. O/N 24/P11/Q8/a,b

8(a)	1 mark for each bullet point (max 2) - To store the value of flags/bits ... that can be changed/set/cleared after arithmetic / logical operations - To allow flags to be checked ... to change instruction sequence	2
8(b)	1 mark for each bullet point (max 2) - Special purpose registers have a specified role in the machine whereas general purpose registers can be used for all purposes defined by the programmer - Special purpose registers hold the state of the program's execution while general purpose registers hold the program's data during operations	2

16. O/N 24/P12/Q3/a,b

3(a)	1 mark for each correct purpose	4										
<table><tr><th>Register</th><th>Purpose</th></tr><tr><td>Program Counter (PC)</td><td>Stores the address of the next instruction to be fetched/executed</td></tr><tr><td>Memory Address Register (MAR)</td><td>Stores the address of the memory location where data will be read from/written to</td></tr><tr><td>Memory Data Register (MDR)</td><td>Stores the data read from the address in the MAR // stores the data to be written to the address in the MAR</td></tr><tr><td>Index Register (IX)</td><td>Stores a number that will be added to the operand, to form the address of the data</td></tr></table>			Register	Purpose	Program Counter (PC)	Stores the address of the next instruction to be fetched/executed	Memory Address Register (MAR)	Stores the address of the memory location where data will be read from/written to	Memory Data Register (MDR)	Stores the data read from the address in the MAR // stores the data to be written to the address in the MAR	Index Register (IX)	Stores a number that will be added to the operand, to form the address of the data
Register	Purpose											
Program Counter (PC)	Stores the address of the next instruction to be fetched/executed											
Memory Address Register (MAR)	Stores the address of the memory location where data will be read from/written to											
Memory Data Register (MDR)	Stores the data read from the address in the MAR // stores the data to be written to the address in the MAR											
Index Register (IX)	Stores a number that will be added to the operand, to form the address of the data											
3(b)	1 mark for each bullet point (max 4) e.g. • HDMI has faster transfer rates than VGA • ... needed due to high resolution / large number of pixels of monitor // HDMI supports the high resolution of the monitor • HDMI supports video and audio transfer between computer and monitor speakers • ... so no separate sound cable is needed unlike VGA • HDMI is digital interface therefore no data is lost in transfer to analogue and back • HDMI is less prone to error/crosstalk/external interference	4										

17. O/N 24/P12/Q8/a,b

8(a)	1 mark for each correct content of the ACC (4) 1 mark for correct IX column <table><thead><tr><th></th><th>Instructions</th><th>ACC content</th><th>IX content</th></tr></thead><tbody><tr><td>1</td><td>LDM #19 DEC ACC</td><td>18</td><td>20</td></tr><tr><td>2</td><td>LDD 23 ADD 19</td><td>40</td><td>20</td></tr><tr><td>3</td><td>LDI 25 INC ACC</td><td>5</td><td>20</td></tr><tr><td>4</td><td>LDR #21 LDX 2</td><td>15</td><td>21</td></tr></tbody></table>		Instructions	ACC content	IX content	1	LDM #19 DEC ACC	18	20	2	LDD 23 ADD 19	40	20	3	LDI 25 INC ACC	5	20	4	LDR #21 LDX 2	15	21	5
	Instructions	ACC content	IX content																			
1	LDM #19 DEC ACC	18	20																			
2	LDD 23 ADD 19	40	20																			
3	LDI 25 INC ACC	5	20																			
4	LDR #21 LDX 2	15	21																			
8(b)(i)	1 mark each correct content of ACC <table><thead><tr><th></th><th>Instructions</th><th>ACC content</th></tr></thead><tbody><tr><td>1</td><td>LSL #2</td><td>0110 1000</td></tr><tr><td>2</td><td>ADD #5 AND #30</td><td>0001 1110</td></tr><tr><td>3</td><td>OR B11110010 INC ACC</td><td>1111 1011</td></tr></tbody></table>		Instructions	ACC content	1	LSL #2	0110 1000	2	ADD #5 AND #30	0001 1110	3	OR B11110010 INC ACC	1111 1011	3								
	Instructions	ACC content																				
1	LSL #2	0110 1000																				
2	ADD #5 AND #30	0001 1110																				
3	OR B11110010 INC ACC	1111 1011																				
8(b)(ii)	1 mark each to max 2 for explanation - An odd binary number will have a 1 in the Least Significant Bit (LSB) - A bit manipulation operation is required to access/mask only the LSB and clear all the others - Compare the result of the masking with denary 1 ... the result of the comparison will be true if the number is odd 1 mark for correct instruction - AND B00000001 // AND #1 // AND &01	3																				

18. O/N 24/P13/Q7/a,b

7(a)	1 mark each: <table border="1" data-bbox="284 282 1046 730"> <thead> <tr> <th></th><th>Instructions</th><th>ACC content</th></tr> </thead> <tbody> <tr> <td>1</td><td>LDD 50 ADD #4 ADD 54</td><td>158</td></tr> <tr> <td>2</td><td>LDI 53 DEC ACC ADD 56</td><td>99</td></tr> <tr> <td>3</td><td>LDM #55 SUB #5</td><td>50</td></tr> </tbody> </table>		Instructions	ACC content	1	LDD 50 ADD #4 ADD 54	158	2	LDI 53 DEC ACC ADD 56	99	3	LDM #55 SUB #5	50	3
	Instructions	ACC content												
1	LDD 50 ADD #4 ADD 54	158												
2	LDI 53 DEC ACC ADD 56	99												
3	LDM #55 SUB #5	50												
7(b)	1 mark for each bullet point for the explanation 1 mark for correct instruction • A bit manipulation operation is required to set all the bits to zero • Compare the result of the masking with 0 • ... the result of comparison will be true if the register is cleared • AND B00000000 / #00 / &00	3												

19. O/N 24/P13/Q8/c

8(c)	1 mark each: • A logical shift moves all bits to the right and inserts zeros in the appropriate leftmost bits • An arithmetic shift moves all bits to the right but copies the sign bit into the Most Significant Bit (MSB)	2
------	---	----------

20. M/J 24/P11/Q4/a,b

4(a)	1 mark for each correct answer: <table border="1" data-bbox="284 1552 861 2074"> <thead> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>LDD 20 ADD #2</td><td>4</td></tr> <tr> <td>2</td><td>LDX 22</td><td>5</td></tr> <tr> <td>3</td><td>LDI 25 INC ACC SUB 22</td><td>1</td></tr> <tr> <td>4</td><td>LDD 19 LDM #5 LDM #25</td><td>25</td></tr> </tbody> </table>	Program Number	Code	ACC Content	1	LDD 20 ADD #2	4	2	LDX 22	5	3	LDI 25 INC ACC SUB 22	1	4	LDD 19 LDM #5 LDM #25	25	4
Program Number	Code	ACC Content															
1	LDD 20 ADD #2	4															
2	LDX 22	5															
3	LDI 25 INC ACC SUB 22	1															
4	LDD 19 LDM #5 LDM #25	25															

4(b)	1 mark for each correct answer: <table border="1" data-bbox="284 237 951 533"> <thead> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>AND 31</td><td>1001 1010</td></tr> <tr> <td>2</td><td>XOR B01001111</td><td>1101 0101</td></tr> <tr> <td>3</td><td>OR #30</td><td>1001 1110</td></tr> </tbody> </table>	Program Number	Code	ACC Content	1	AND 31	1001 1010	2	XOR B01001111	1101 0101	3	OR #30	1001 1110	3
Program Number	Code	ACC Content												
1	AND 31	1001 1010												
2	XOR B01001111	1101 0101												
3	OR #30	1001 1110												

21. M/J 24/P12/Q5/a,b

5(a)	1 mark for each correct answer: <table border="1" data-bbox="284 714 1123 1279"> <thead> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>LDI 15 SUB #1</td><td>11</td></tr> <tr> <td>2</td><td>LDD 14 ADD 11</td><td>13</td></tr> <tr> <td>3</td><td>LDM #11 ADD #3 SUB 16</td><td>2</td></tr> <tr> <td>4</td><td>LDR #2 LDX 14 ADD #2</td><td>14</td></tr> </tbody> </table>	Program Number	Code	ACC Content	1	LDI 15 SUB #1	11	2	LDD 14 ADD 11	13	3	LDM #11 ADD #3 SUB 16	2	4	LDR #2 LDX 14 ADD #2	14	4
Program Number	Code	ACC Content															
1	LDI 15 SUB #1	11															
2	LDD 14 ADD 11	13															
3	LDM #11 ADD #3 SUB 16	2															
4	LDR #2 LDX 14 ADD #2	14															
5(b)	1 mark for each correct answer: <table border="1" data-bbox="284 1386 1123 1686"> <thead> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>XOR 29</td><td>0100 1001</td></tr> <tr> <td>2</td><td>AND #29</td><td>0000 0100</td></tr> <tr> <td>3</td><td>OR B11111111</td><td>1111 1111</td></tr> </tbody> </table>	Program Number	Code	ACC Content	1	XOR 29	0100 1001	2	AND #29	0000 0100	3	OR B11111111	1111 1111	3			
Program Number	Code	ACC Content															
1	XOR 29	0100 1001															
2	AND #29	0000 0100															
3	OR B11111111	1111 1111															

22. M/J 24/P13/Q3/a,b

3(a)	1 mark for each correct answer: <table border="1" data-bbox="284 280 842 869"> <thead> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>LDM #50 INC ACC SUB #1</td><td>50</td></tr> <tr> <td>2</td><td>LDI 51 ADD 52</td><td>97</td></tr> <tr> <td>3</td><td>LDR #2 LDX 50 DEC ACC</td><td>48</td></tr> <tr> <td>4</td><td>LDD 52 SUB 54 INC ACC</td><td>44</td></tr> </tbody> </table>	Program Number	Code	ACC Content	1	LDM #50 INC ACC SUB #1	50	2	LDI 51 ADD 52	97	3	LDR #2 LDX 50 DEC ACC	48	4	LDD 52 SUB 54 INC ACC	44	4
Program Number	Code	ACC Content															
1	LDM #50 INC ACC SUB #1	50															
2	LDI 51 ADD 52	97															
3	LDR #2 LDX 50 DEC ACC	48															
4	LDD 52 SUB 54 INC ACC	44															
3(b)	1 mark for each correct answer: <table border="1" data-bbox="284 981 1008 1276"> <thead> <tr> <th>Instruction Number</th><th>Instruction</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>LSL #2</td><td>1111 1100</td></tr> <tr> <td>2</td><td>XOR 100</td><td>1111 0010</td></tr> <tr> <td>3</td><td>AND 103</td><td>0011 0111</td></tr> </tbody> </table>	Instruction Number	Instruction	ACC Content	1	LSL #2	1111 1100	2	XOR 100	1111 0010	3	AND 103	0011 0111	3			
Instruction Number	Instruction	ACC Content															
1	LSL #2	1111 1100															
2	XOR 100	1111 0010															
3	AND 103	0011 0111															

23. O/N 23/P11/Q5/a,b,c

5(a)	1 mark for identification of the register and 1 mark for role (max 2 for each register) • Program Counter (PC) • stores the <u>address</u> where the <u>next</u> instruction is to be read from • Memory Address Register (MAR) • stores the address of the memory location (or an I/O component) currently being read from or written to • Current Instruction Register (CIR) • holds the instruction currently being decoded and/or executed • Status Register • Contains bits which can be referenced individually and set or cleared depending on the operation e.g. overflow, underflow	4
------	--	----------

5(b)	1 mark for each bullet point (max 2) - Immediate Access Store holds all the data / instructions / programs currently in use - Immediate Access Store is volatile memory - Immediate Access Store has fast access times	2
5(c)(i)	1 mark for each bullet point (max 1) - The CPU can now perform nearly twice as many F-E cycles per second - Instead of 2.1 billion F-E cycles per second, the CPU can now perform 4 billion FE cycles per second	1
5(c)(ii)	1 mark for each bullet point (max 5) - Multiple cores introduce additional overheads ...because of the need for communication between cores - Software may not be designed for multiple cores... ...so one of the cores will be left idle - Memory access speed may not match speed of cores... ...so causing delay - The two computers may have more differences than just the cores ...one may have more RAM which allows faster multitasking ...one may have a GPU ...etc.	5

24. O/N 23/P11/Q8/a,b,c

8(a)	1 mark for: To create a symbol table	1
8(b)(i)	1 mark for each bullet point - Data movement: e.g. LDR #50 // STO 201 - Arithmetic operation: e.g. ADD 100 // INC IX - Conditional instruction: e.g. JPE 96	3
8(b)(ii)	1 mark for each bullet point (max 2) Similarity: - both load the contents of an <u>address</u> into the <u>Accumulator</u> Difference: - direct accesses the address given by the operand whereas indexed adds the contents of IX to the operand and accesses the data at that calculated address	2
8(b)(iii)	1 mark for - Indirect (addressing) - Relative (addressing)	1
8(c)(i)	0000 0101	1

8(c)(ii)	0000 0010	1
8(c)(iii)	1101 0010	1

25. O/N 23/P12/Q8/c

8(c)(i)	1 mark for each bullet point (max 1) • USB / Universal Serial Bus • HDMI	1
8(c)(ii)	1 mark for each component (max 3) Buffers • A buffer temporarily holds data until it is ready to be transmitted to the device Address Bus • The address of the data to be written to the device (in RAM) is carried on the address bus Data Bus • All data to be written to the device / buffer is carried on the data bus	3

26. O/N 23/P12/Q9/a,b

9(a)	1 mark for each bullet point • IN - Input and output of data • ADD - Arithmetic operations • JPE - Unconditional and conditional instructions • CMI - Compare instructions	4
------	---	----------

9(b)

1 mark for each set of shaded rows

4

Instruction address	ACC	IX	Memory address				Output
			100	101	110	111	
			0	0	51	65	
10		0					
11	49						
12				49			
13	51						
14							
15							
16	49						
17	65						
18		1					
19			65				
20							
13	65						
14							
15							
21							A
22							

27. O/N 23/P13/Q7/d

7(d)	1 mark for naming a correct port 2 marks for matching justification	3
	• USB • ...has fast data transfer speeds for data (to the headset) • ...is a universal/popular cable // universal standard • HDMI • ...allows video and audio to be transferred (on the same cable) • ...convenience of HDMI as no need for two cables	

28. O/N 23/P13/Q9/a,b,c,d

9(a)(i)	1 mark for each bullet point (max 3).	3
	• Control unit (CU) • Arithmetic and Logic Unit (ALU) • Immediate Access Store (IAS) • (System clock)	

9(a)(ii)	1 mark for each bullet point (max 2). - Special purpose registers hold the status of a program whereas general purpose registers hold the temporary data while performing operations. - Special purpose registers are specialised for a specific use, whereas general purpose registers are used for any purpose. - General purpose registers can be used by most instructions, whereas special purpose can only be used by certain instruction	2										
9(b)	1 mark for each correct answer (shaded cells) <table><tr><th>Step</th><th>Description</th></tr><tr><td>$PC \leftarrow [PC] + 1$</td><td>The address in PC is incremented.</td></tr><tr><td>$MDR \leftarrow [[MAR]]$</td><td>The data in the location pointed to by the MAR is copied to the MDR.</td></tr><tr><td>$MAR \leftarrow [PC]$</td><td>The contents of PC are copied to the MAR.</td></tr><tr><td>$CIR \leftarrow [MDR]$</td><td>The contents of MDR are copied into CIR.</td></tr></table>	Step	Description	$PC \leftarrow [PC] + 1$	The address in PC is incremented.	$MDR \leftarrow [[MAR]]$	The data in the location pointed to by the MAR is copied to the MDR.	$MAR \leftarrow [PC]$	The contents of PC are copied to the MAR.	$CIR \leftarrow [MDR]$	The contents of MDR are copied into CIR.	4
Step	Description											
$PC \leftarrow [PC] + 1$	The address in PC is incremented.											
$MDR \leftarrow [[MAR]]$	The data in the location pointed to by the MAR is copied to the MDR.											
$MAR \leftarrow [PC]$	The contents of PC are copied to the MAR.											
$CIR \leftarrow [MDR]$	The contents of MDR are copied into CIR.											
9(c)	1 mark for an appropriate example. Examples: - Division by zero - Runtime error - Out of memory bounds - Program requesting an external device / input - Buffer overflow	1										
9(d)	1 mark for each letter in the correct position (2, 4 and 6) D A B	3										

29. M/J 23/P11/Q3/d(vi)

3(d)(vi)	00011001	1
----------	----------	----------

30. M/J 23/P11/Q6

6	1 mark each to max 5 • An interrupt flag is raised in the (interrupt) register • At the end of the current FE cycle // at the start of the next FE cycle • The system checks the interrupt register for higher priority interrupts than current process • If true, it stores the current contents of the registers on the stack • The appropriate interrupt service routine (ISR) for the key press is called • The input data from the keyboard is processed • The contents of the registers are restored from the stack • ... and control is passed back to previous process	5
----------	---	----------

31. M/J 23/P12/Q3/a,b

3(a)	1 mark for generates object code to second pass 1 mark for reads source code one line at a time to both boxes 1 mark for removes white space and adds labels to first pass Action generates object code reads the source code one line at a time removes white space adds labels to the symbol table Pass first second <pre> graph LR subgraph Action A1[generates object code] A2[reads the source code one line at a time] A3[removes white space] A4[adds labels to the symbol table] end subgraph Pass P1[first] P2[second] end A1 --> P2 A2 --> P1 A2 --> P2 A3 --> P1 A4 --> P1 </pre>	3
3(b)	1 mark for each correct term Direct addressing is when the operand holds the memory address of the data. Indirect addressing is when the operand holds a memory address that stores the memory address of the data. Immediate addressing is when the operand is the data.	3

32. M/J 23/P12/Q4/d

4(d)	0011 1100	1
------	-----------	----------

33. M/J 23/P12/Q5/a,b,c

5(a)	1 mark each to max 2 Examples: • Interrupt • Timing • Read • Write	2
5(b)	1 mark for description; 1 mark for corresponding explanation Examples • Increase number of cores • Each core can independently carry out a process at the same time // so that more instructions are performed in parallel • Increase RAM capacity • ... allowing more applications to reside in memory at the same time, saving disk access times • Increase cache memory • More data can be stored in fast access so less time is spent accessing from RAM • Increase clock speed • More Fetch-Decode-Execute (FDE) cycles can run each second / per unit time	4
5(c)(i)	1 mark for a correct answer • 1 bit is transferred at a time • Can be synchronous or asynchronous • USB-3 is full duplex and earlier versions are half-duplex	1
5(c)(ii)	1 mark for identification of a suitable port Examples • HDMI • DisplayPort	1

34. M/J 23/P13/Q7/b,c

7(b)	1 mark each to max 4 • The system clock gives out timing signals • ... which are sent on the control bus • ...to synchronise the other system components • The Control Unit initiates data transfer • ...by generating signals that are sent on the control bus to other components	4						
7(c)	1 mark for each Register transfer notation <table><tr><th>Stage description</th><th>Register transfer notation</th></tr><tr><td>The Program Counter (PC) is incremented</td><td>$PC \leftarrow [PC] + 1$</td></tr><tr><td>The data in the address stored in the Memory Address Register (MAR) is copied to the Memory Data Register (MDR)</td><td>$MDR \leftarrow [[MAR]]$</td></tr></table>	Stage description	Register transfer notation	The Program Counter (PC) is incremented	$PC \leftarrow [PC] + 1$	The data in the address stored in the Memory Address Register (MAR) is copied to the Memory Data Register (MDR)	$MDR \leftarrow [[MAR]]$	2
Stage description	Register transfer notation							
The Program Counter (PC) is incremented	$PC \leftarrow [PC] + 1$							
The data in the address stored in the Memory Address Register (MAR) is copied to the Memory Data Register (MDR)	$MDR \leftarrow [[MAR]]$							

35. O/N 22/P11/Q5/a,b,c,d

5(a)	Instructions and data are stored in the same memory space / in main memory.	1												
5(b)(i)	1 mark for each special purpose register: Program Counter (PC): - to store the address / location / memory location of the <u>next</u> instruction to be fetched Index Register (IX): - to store a value that is added to an address to give another address Status Register (SR): - to store flags which are set by events // from the results of arithmetic and logic operations and interrupt flags	3												
5(b)(ii)	1 mark for both rows: <table><tr><th>CPU component</th><th>Data bus</th><th>Address bus</th><th>Control bus</th></tr><tr><td>System clock</td><td></td><td></td><td>✓</td></tr><tr><td>Memory Address Register (MAR)</td><td></td><td>✓</td><td></td></tr></table>	CPU component	Data bus	Address bus	Control bus	System clock			✓	Memory Address Register (MAR)		✓		1
CPU component	Data bus	Address bus	Control bus											
System clock			✓											
Memory Address Register (MAR)		✓												

5(b)(iii)	1 mark for each bullet point (max 2): • to coordinate / synchronise the actions of other components in the CPU • to send / receive control signals along the control bus • to manage the execution of instructions (in sequence) • to control the communication between the components of the CPU	2
5(c)	1 mark for each bullet point: • to send a signal from a device or process • ... seeking the attention of the processor	2
5(d)	1 mark for each bullet point (max 2). For example: • division by zero // runtime error in a program • attempt to access an invalid memory location • array index out of bounds • stack overflow	2

36. O/N 22/P11/Q6/a,b,c

6(a)(i)	1 mark for each set of highlighted rows.	4																																																																																																																																																																																																																																																																	
	<table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th rowspan="2">IX</th><th colspan="5">Memory address</th><th rowspan="2">Output</th></tr><tr><th>100</th><th>101</th><th>110</th><th>111</th><th>112</th></tr><tr><td></td><td></td><td></td><td>0</td><td>0</td><td>66</td><td>65</td><td>35</td><td></td></tr><tr><td>77</td><td></td><td>0</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>78</td><td>66</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>79</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>80</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>81</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>82</td><td></td><td></td><td></td><td>66</td><td></td><td></td><td></td><td></td></tr><tr><td>83</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>84</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>85</td><td></td><td></td><td>1</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>86</td><td></td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>87</td><td>65</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>88</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>89</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>81</td><td>66</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>82</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>83</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>84</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>85</td><td></td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>86</td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>87</td><td>35</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>88</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>89</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>90</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>91</td><td>50</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>92</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td>2</td></tr><tr><td>93</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Instruction address	ACC	IX	Memory address					Output	100	101	110	111	112				0	0	66	65	35		77		0							78	66								79									80									81									82				66					83	1								84									85			1						86		1							87	65								88									89									81	66								82									83	1								84	2								85			2						86		2							87	35								88									89									90	2								91	50								92								2	93									
Instruction address	ACC				IX	Memory address					Output																																																																																																																																																																																																																																																								
		100	101	110		111	112																																																																																																																																																																																																																																																												
			0	0	66	65	35																																																																																																																																																																																																																																																												
77		0																																																																																																																																																																																																																																																																	
78	66																																																																																																																																																																																																																																																																		
79																																																																																																																																																																																																																																																																			
80																																																																																																																																																																																																																																																																			
81																																																																																																																																																																																																																																																																			
82				66																																																																																																																																																																																																																																																															
83	1																																																																																																																																																																																																																																																																		
84																																																																																																																																																																																																																																																																			
85			1																																																																																																																																																																																																																																																																
86		1																																																																																																																																																																																																																																																																	
87	65																																																																																																																																																																																																																																																																		
88																																																																																																																																																																																																																																																																			
89																																																																																																																																																																																																																																																																			
81	66																																																																																																																																																																																																																																																																		
82																																																																																																																																																																																																																																																																			
83	1																																																																																																																																																																																																																																																																		
84	2																																																																																																																																																																																																																																																																		
85			2																																																																																																																																																																																																																																																																
86		2																																																																																																																																																																																																																																																																	
87	35																																																																																																																																																																																																																																																																		
88																																																																																																																																																																																																																																																																			
89																																																																																																																																																																																																																																																																			
90	2																																																																																																																																																																																																																																																																		
91	50																																																																																																																																																																																																																																																																		
92								2																																																																																																																																																																																																																																																											
93																																																																																																																																																																																																																																																																			
6(a)(ii)	swaps the contents of memory address <u>100</u> and <u>101</u>	1																																																																																																																																																																																																																																																																	
6(b)(i)	1000 1100	1																																																																																																																																																																																																																																																																	

6(b)(ii)	1001 0000	1															
6(b)(iii)	1101 1111	1															
6(b)(iv)	0010 0100	1															
6(c)	1 mark for each pair of highlighted rows <table> <tr> <th>Task</th><th>First pass</th><th>Second pass</th></tr> <tr> <td>Remove comments.</td><td>✓</td><td></td></tr> <tr> <td>Read the assembly language program one line at a time.</td><td>✓</td><td>✓</td></tr> <tr> <td>Generate the object code.</td><td></td><td>✓</td></tr> <tr> <td>Check the opcode is in the instruction set.</td><td>✓</td><td></td></tr> </table>	Task	First pass	Second pass	Remove comments.	✓		Read the assembly language program one line at a time.	✓	✓	Generate the object code.		✓	Check the opcode is in the instruction set.	✓		2
Task	First pass	Second pass															
Remove comments.	✓																
Read the assembly language program one line at a time.	✓	✓															
Generate the object code.		✓															
Check the opcode is in the instruction set.	✓																

37. O/N 22/P11/Q7/d,e

7(d)	1 mark for each bullet point (max 2): • cache is fast access memory (close to the CPU) • cache stores frequently used instructions / data • ... more cache means more instructions / data can be transferred faster • ... less swapping between RAM and cache • prevents the CPU idling while waiting for data	2
7(e)	1 mark for each device. • 3D printer: USB port / COM port • Monitor: HDMI / VGA / USB / DisplayPort	2

38. O/N 22/P12/Q7/a,b,c

7(a)	1 mark for each set of highlighted rows.										5																																																																																																																																																																																																																																																																																																																											
<table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th rowspan="2">IX</th><th colspan="7">Memory address</th></tr><tr><th>100</th><th>101</th><th>102</th><th>103</th><th>110</th><th>111</th><th>112</th></tr><tr><td></td><td></td><td></td><td>0</td><td>0</td><td>112</td><td>4</td><td>1</td><td>4</td><td>0</td><td></td></tr><tr><td>75</td><td></td><td>0</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>76</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>77</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>78</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>79</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>80</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>84</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>85</td><td></td><td></td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>86</td><td>0</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>87</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>88</td><td></td><td></td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>89</td><td></td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>90</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>76</td><td>4</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>77</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>78</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>79</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>80</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>81</td><td>6</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>82</td><td></td><td></td><td></td><td>6</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>83</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>86</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>87</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>88</td><td></td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>89</td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>90</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>												Instruction address	ACC	IX	Memory address							100	101	102	103	110	111	112				0	0	112	4	1	4	0		75		0									76	1										77											78											79											80											84	2										85				2							86	0										87	1										88			1								89		1									90											76	4										77											78											79											80											81	6										82				6							83											86	1										87	2										88			2								89		2									90											
Instruction address	ACC	IX	Memory address																																																																																																																																																																																																																																																																																																																																			
			100	101	102	103	110	111	112																																																																																																																																																																																																																																																																																																																													
			0	0	112	4	1	4	0																																																																																																																																																																																																																																																																																																																													
75		0																																																																																																																																																																																																																																																																																																																																				
76	1																																																																																																																																																																																																																																																																																																																																					
77																																																																																																																																																																																																																																																																																																																																						
78																																																																																																																																																																																																																																																																																																																																						
79																																																																																																																																																																																																																																																																																																																																						
80																																																																																																																																																																																																																																																																																																																																						
84	2																																																																																																																																																																																																																																																																																																																																					
85				2																																																																																																																																																																																																																																																																																																																																		
86	0																																																																																																																																																																																																																																																																																																																																					
87	1																																																																																																																																																																																																																																																																																																																																					
88			1																																																																																																																																																																																																																																																																																																																																			
89		1																																																																																																																																																																																																																																																																																																																																				
90																																																																																																																																																																																																																																																																																																																																						
76	4																																																																																																																																																																																																																																																																																																																																					
77																																																																																																																																																																																																																																																																																																																																						
78																																																																																																																																																																																																																																																																																																																																						
79																																																																																																																																																																																																																																																																																																																																						
80																																																																																																																																																																																																																																																																																																																																						
81	6																																																																																																																																																																																																																																																																																																																																					
82				6																																																																																																																																																																																																																																																																																																																																		
83																																																																																																																																																																																																																																																																																																																																						
86	1																																																																																																																																																																																																																																																																																																																																					
87	2																																																																																																																																																																																																																																																																																																																																					
88			2																																																																																																																																																																																																																																																																																																																																			
89		2																																																																																																																																																																																																																																																																																																																																				
90																																																																																																																																																																																																																																																																																																																																						
7(b)(i)	0100 1100										1																																																																																																																																																																																																																																																																																																																											
7(b)(ii)	0100 0001										1																																																																																																																																																																																																																																																																																																																											

7(b)(iii)	1001 1000	1								
7(b)(iv)	1101 1111	1								
7(c)	1 mark for each correct row: <table><thead><tr><th>Description</th><th>Register transfer notation</th></tr></thead><tbody><tr><td>Copy the address of the next instruction into the Memory Address Register.</td><td>MAR ← [PC]</td></tr><tr><td>Increment the Program Counter.</td><td>PC ← [PC] + 1</td></tr><tr><td>Copy the contents of the Memory Data Register into the Current Instruction Register.</td><td>CIR ← [MDR]</td></tr></tbody></table>	Description	Register transfer notation	Copy the address of the next instruction into the Memory Address Register.	MAR ← [PC]	Increment the Program Counter.	PC ← [PC] + 1	Copy the contents of the Memory Data Register into the Current Instruction Register.	CIR ← [MDR]	3
Description	Register transfer notation									
Copy the address of the next instruction into the Memory Address Register.	MAR ← [PC]									
Increment the Program Counter.	PC ← [PC] + 1									
Copy the contents of the Memory Data Register into the Current Instruction Register.	CIR ← [MDR]									

39. O/N 22/P13/Q4/a,b,c

4(a)(i)	1 mark for each register: MAR: holds address in memory from which data will be read / to which data will be written MDR: holds the data/instructions which has been read from or is to be written to the address in the MAR	2
4(a)(ii)	after completion of the execute stage // before the cycle begins	1
4(b)	1 mark for each bullet point (max 2): - synchronise operations ... by creating timing signals - to keep track of the date and time / timestamp files - to process operations in the correct order / sequence	2
4(c)	1 mark for identification of a correct upgrade: 1 mark for a corresponding explanation: Examples: - increase quantity of RAM ... so allowing more applications to reside in memory at the same time, saving disk access times - increase the size of cache memory ... so that the CPU can continue working without waiting for data - increase clock speed ... so that more instructions are performed in a time period - increase the number of processors / cores ... so that more instructions are performed in parallel	2

40. O/N 22/P13/Q6/a,b,c

6(a)(i)	1 mark for each set of highlighted rows: <table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th rowspan="2">IX</th><th colspan="5">Memory address</th><th rowspan="2">Output</th></tr><tr><th>100</th><th>101</th><th>110</th><th>111</th><th>112</th></tr><tr><td></td><td></td><td></td><td>1</td><td>0</td><td>97</td><td>98</td><td>97</td><td></td></tr><tr><td>75</td><td></td><td>0</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>76</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>77</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>78</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>79</td><td>97</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>80</td><td>65</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>81</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>82</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>83</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>84</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>85</td><td></td><td></td><td></td><td>1</td><td></td><td></td><td></td><td></td></tr><tr><td>86</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>87</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>88</td><td></td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>89</td><td></td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>90</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>76</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>77</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>78</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>91</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>92</td><td>49</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>93</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td>1</td></tr><tr><td>94</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Instruction address	ACC	IX	Memory address					Output	100	101	110	111	112				1	0	97	98	97		75		0							76	1								77									78									79	97								80	65								81									82									83	1								84									85				1					86									87	2								88			2						89		1							90									76									77									78									91	1								92	49								93								1	94									4
Instruction address	ACC				IX	Memory address					Output																																																																																																																																																																																																																													
		100	101	110		111	112																																																																																																																																																																																																																																	
			1	0	97	98	97																																																																																																																																																																																																																																	
75		0																																																																																																																																																																																																																																						
76	1																																																																																																																																																																																																																																							
77																																																																																																																																																																																																																																								
78																																																																																																																																																																																																																																								
79	97																																																																																																																																																																																																																																							
80	65																																																																																																																																																																																																																																							
81																																																																																																																																																																																																																																								
82																																																																																																																																																																																																																																								
83	1																																																																																																																																																																																																																																							
84																																																																																																																																																																																																																																								
85				1																																																																																																																																																																																																																																				
86																																																																																																																																																																																																																																								
87	2																																																																																																																																																																																																																																							
88			2																																																																																																																																																																																																																																					
89		1																																																																																																																																																																																																																																						
90																																																																																																																																																																																																																																								
76																																																																																																																																																																																																																																								
77																																																																																																																																																																																																																																								
78																																																																																																																																																																																																																																								
91	1																																																																																																																																																																																																																																							
92	49																																																																																																																																																																																																																																							
93								1																																																																																																																																																																																																																																
94																																																																																																																																																																																																																																								
6(a)(ii)	1 mark for each bullet point: To allow for re-locatable code ... because all (target) addresses can be specified by the base address + offset	2																																																																																																																																																																																																																																						
6(b)(i)	0000 0100	1																																																																																																																																																																																																																																						

6(b)(ii)	1101 1111	1
6(b)(iii)	0010 0111	1
6(c)	1 mark for a correct name: - input and output of data - arithmetic operations - unconditional and conditional instructions - compare instructions	1

41. O/N 21/P11/Q6/a,b

6(a)	1 mark for identification of line and description of error 1 mark for the correct statement			4
	Line number	Description of the error	Correct statement	
	2	Program Counter should be incremented, not decremented	$PC \leftarrow [PC] + 1$	
	3	It should be the contents of the address in the MAR	$MDR \leftarrow [[MAR]]$	
6(b)	1 mark for each correct row			4
	Current contents of the ACC	Instruction	New contents of the ACC	
	11111111	OR 101	11111111	
	00000000	XOR #15	00001111	
	10101010	LSR #2	00101010	
	01010101	AND 104	00000000	

42. O/N 21/P12/Q7/a

7(a)	1 mark per pair of rows (shaded & unshaded)			3
Event		Hardware Interrupt	Software Interrupt	
Buffer full			✓	} 1
Printer is out of paper		✓		
User has pressed a key on the keyboard		✓		} 1
Division by zero			✓	
Power failure		✓		} 1
Stack overflow			✓	

43. O/N 21/P12/Q8/a,b

8(a)(i)	1 mark for each bullet point to max 2 for each register MAR • Stores the next <u>address</u> to be fetched • ... held in the Program Counter (PC) • The data at this address is then fetched MDR • Stores the data from the address pointed to by the MAR • The data in it is copied to the Current Instruction Register (CIR)	4															
8(a)(ii)	1 mark for a correct register e.g. Program Counter (PC) Current Instruction Register (CIR) Status register Interrupt register	1															
8(b)(i)	1 mark for each correct answer <table><tr><th>Current contents of the ACC</th><th>Instruction</th><th>New contents of the ACC</th></tr><tr><td>01010101</td><td>XOR 101</td><td>1010 0101</td></tr><tr><td>11110000</td><td>AND 104</td><td>1111 0000</td></tr><tr><td>00001111</td><td>LSL #4</td><td>1111 0000</td></tr><tr><td>11111111</td><td>OR 102</td><td>1111 1111</td></tr></table>	Current contents of the ACC	Instruction	New contents of the ACC	01010101	XOR 101	1010 0101	11110000	AND 104	1111 0000	00001111	LSL #4	1111 0000	11111111	OR 102	1111 1111	4
Current contents of the ACC	Instruction	New contents of the ACC															
01010101	XOR 101	1010 0101															
11110000	AND 104	1111 0000															
00001111	LSL #4	1111 0000															
11111111	OR 102	1111 1111															

8(b)(ii)	1 mark for each correct instruction <table> <tr> <th>Instruction Group</th> <th>Instruction</th> </tr> <tr> <td>Data movement</td> <td>LDM #2</td> </tr> <tr> <td>Input and output of data</td> <td>IN / OUT</td> </tr> <tr> <td>Arithmetic Operations</td> <td>INC ACC / INC IX</td> </tr> <tr> <td>Unconditional and conditional instructions</td> <td>JPN 100 / JMP 100</td> </tr> <tr> <td>Compare instructions</td> <td>CMP 100</td> </tr> </table>	Instruction Group	Instruction	Data movement	LDM #2	Input and output of data	IN / OUT	Arithmetic Operations	INC ACC / INC IX	Unconditional and conditional instructions	JPN 100 / JMP 100	Compare instructions	CMP 100	4
Instruction Group	Instruction													
Data movement	LDM #2													
Input and output of data	IN / OUT													
Arithmetic Operations	INC ACC / INC IX													
Unconditional and conditional instructions	JPN 100 / JMP 100													
Compare instructions	CMP 100													
8(b)(iii)	1 mark for name, 1 mark for description - Indirect addressing - the address to be used is at the given address - Relative addressing - the address to be used is an offset number of locations away, relative to the address of the current instruction - Indexed addressing - form the address from the given address plus the contents of the index register	2												

44. O/N 21/P13/Q6/a,b

6(a)	1 mark for identification of line and description of error 1 mark for the correct statement <table border="1" data-bbox="288 1312 1305 1608"> <thead> <tr> <th>Line number</th><th>Description of the error</th><th>Correct statement</th></tr> </thead> <tbody> <tr> <td>2</td><td>Program Counter should be incremented, not decremented</td><td>$PC \leftarrow [PC] + 1$</td></tr> <tr> <td>3</td><td>It should be the contents of the address in the MAR</td><td>$MDR \leftarrow [[MAR]]$</td></tr> </tbody> </table>	Line number	Description of the error	Correct statement	2	Program Counter should be incremented, not decremented	$PC \leftarrow [PC] + 1$	3	It should be the contents of the address in the MAR	$MDR \leftarrow [[MAR]]$	4						
Line number	Description of the error	Correct statement															
2	Program Counter should be incremented, not decremented	$PC \leftarrow [PC] + 1$															
3	It should be the contents of the address in the MAR	$MDR \leftarrow [[MAR]]$															
6(b)	1 mark for each correct row <table border="1" data-bbox="288 1720 1305 2040"> <thead> <tr> <th>Current contents of the ACC</th><th>Instruction</th><th>New contents of the ACC</th></tr> </thead> <tbody> <tr> <td>11111111</td><td>OR 101</td><td>11111111</td></tr> <tr> <td>00000000</td><td>XOR #15</td><td>00001111</td></tr> <tr> <td>10101010</td><td>LSR #2</td><td>00101010</td></tr> <tr> <td>01010101</td><td>AND 104</td><td>00000000</td></tr> </tbody> </table>	Current contents of the ACC	Instruction	New contents of the ACC	11111111	OR 101	11111111	00000000	XOR #15	00001111	10101010	LSR #2	00101010	01010101	AND 104	00000000	4
Current contents of the ACC	Instruction	New contents of the ACC															
11111111	OR 101	11111111															
00000000	XOR #15	00001111															
10101010	LSR #2	00101010															
01010101	AND 104	00000000															

45. M/J 21/P11/Q3/a,b,c

3(a)	1 mark for each completed statement The Program Counter holds the address of the next instruction to be loaded. This address is sent to the Memory Address Register. The Memory Data Register holds the data fetched from this address. This data is sent to the Current Instruction Register and the Control Unit decodes the instruction's opcode. The Program Counter is incremented.	5																																																																																																																																																																																																				
3(b)	1 mark for each shaded set of values <table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th colspan="4">Memory address</th><th rowspan="2">IX</th><th rowspan="2">Output</th></tr><tr><th>365</th><th>366</th><th>367</th><th>368</th></tr><tr><td></td><td></td><td>1</td><td>3</td><td>65</td><td>66</td><td>0</td><td></td></tr><tr><td>200</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>201</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>202</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>203</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>204</td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>205</td><td></td><td></td><td></td><td></td><td></td><td>2</td><td></td></tr><tr><td>206</td><td>65</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>207</td><td></td><td></td><td></td><td></td><td></td><td></td><td>A</td></tr><tr><td>208</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>200</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>201</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>202</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>203</td><td>3</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>204</td><td></td><td>3</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>205</td><td></td><td></td><td></td><td></td><td></td><td>3</td><td></td></tr><tr><td>206</td><td>66</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>207</td><td></td><td></td><td></td><td></td><td></td><td></td><td>B</td></tr><tr><td>208</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>200</td><td>3</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>201</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>202</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>209</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Instruction address	ACC	Memory address				IX	Output	365	366	367	368			1	3	65	66	0		200	1							201								202								203	2							204		2						205						2		206	65							207							A	208								200	2							201								202								203	3							204		3						205						3		206	66							207							B	208								200	3							201								202								209								6
Instruction address	ACC			Memory address						IX	Output																																																																																																																																																																																											
		365	366	367	368																																																																																																																																																																																																	
		1	3	65	66	0																																																																																																																																																																																																
200	1																																																																																																																																																																																																					
201																																																																																																																																																																																																						
202																																																																																																																																																																																																						
203	2																																																																																																																																																																																																					
204		2																																																																																																																																																																																																				
205						2																																																																																																																																																																																																
206	65																																																																																																																																																																																																					
207							A																																																																																																																																																																																															
208																																																																																																																																																																																																						
200	2																																																																																																																																																																																																					
201																																																																																																																																																																																																						
202																																																																																																																																																																																																						
203	3																																																																																																																																																																																																					
204		3																																																																																																																																																																																																				
205						3																																																																																																																																																																																																
206	66																																																																																																																																																																																																					
207							B																																																																																																																																																																																															
208																																																																																																																																																																																																						
200	3																																																																																																																																																																																																					
201																																																																																																																																																																																																						
202																																																																																																																																																																																																						
209																																																																																																																																																																																																						

3(c)(i)	<table><tr><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	0	1	0	1	0	0	1
1	1	0	1	0	1	0	0			
3(c)(ii)	1 mark for correct answer The number is divided by 8 (and only whole number retained)	1								

46. M/J 21/P12/Q4/a,b(ii,iii)

4(a)	1 mark for each shaded section / bullet point - Load 65 into ACC - Load 100 into ACC, increment and store in 102 - Load 68 into ACC - Load 101 into ACC, decrement and store in 102 <table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th colspan="3">Memory address</th></tr><tr><th>100</th><th>101</th><th>102</th></tr><tr><td></td><td></td><td>68</td><td>65</td><td>100</td></tr><tr><td>70</td><td>65</td><td></td><td></td><td></td></tr><tr><td>71</td><td></td><td></td><td></td><td></td></tr><tr><td>72</td><td></td><td></td><td></td><td></td></tr><tr><td>73</td><td></td><td></td><td></td><td></td></tr><tr><td>74</td><td></td><td></td><td></td><td></td></tr><tr><td>76</td><td>100</td><td></td><td></td><td></td></tr><tr><td>77</td><td>101</td><td></td><td></td><td></td></tr><tr><td>78</td><td></td><td></td><td></td><td>101</td></tr><tr><td>79</td><td></td><td></td><td></td><td></td></tr><tr><td>70</td><td>68</td><td></td><td></td><td></td></tr><tr><td>71</td><td></td><td></td><td></td><td></td></tr><tr><td>72</td><td></td><td></td><td></td><td></td></tr><tr><td>80</td><td>101</td><td></td><td></td><td></td></tr><tr><td>81</td><td>100</td><td></td><td></td><td></td></tr><tr><td>82</td><td></td><td></td><td></td><td>100</td></tr><tr><td>83</td><td></td><td></td><td></td><td></td></tr><tr><td>(70)</td><td></td><td></td><td></td><td></td></tr></table>	Instruction address	ACC	Memory address			100	101	102			68	65	100	70	65				71					72					73					74					76	100				77	101				78				101	79					70	68				71					72					80	101				81	100				82				100	83					(70)					4
Instruction address	ACC			Memory address																																																																																																
		100	101	102																																																																																																
		68	65	100																																																																																																
70	65																																																																																																			
71																																																																																																				
72																																																																																																				
73																																																																																																				
74																																																																																																				
76	100																																																																																																			
77	101																																																																																																			
78				101																																																																																																
79																																																																																																				
70	68																																																																																																			
71																																																																																																				
72																																																																																																				
80	101																																																																																																			
81	100																																																																																																			
82				100																																																																																																
83																																																																																																				
(70)																																																																																																				
4(b)(ii)	AND	1																																																																																																		
4(b)(iii)	1 mark for AND, 1 mark for #15 AND #15	2																																																																																																		

47. M/J 21/P12/Q5/a,b

5(a)	1 mark for each term correctly inserted The control unit/bus transmits the signals to coordinate events based on the pulses of the (system) clock. The data bus carries data to components, while the address bus carries the address where data is being written to or read from. The arithmetic logic unit/ALU performs mathematical operations and logical comparisons.	5
5(b)	1 mark per bullet point to max 3 per factor. max 4 overall. Number of cores: • Each core processes one <u>instruction</u> per clock pulse • More/multiple cores mean that sequences of instructions can be split between them • ... and so more than one <u>instruction</u> is executed per clock pulse // more sequences of instructions can be run at the same time • More cores decreases the time taken to complete task Clock speed: • Each <u>instruction</u> is executed on a clock pulse // one F-E cycle is run on each clock pulse • ... so the clock speed dictates the number of <u>instructions</u> that can be run per second • The faster the clock speed the more <u>instructions</u> can be run per second	4

48. M/J 21/P13/Q3/a,b,c

3(a)	1 mark for each completed statement The Program Counter holds the address of the next instruction to be loaded. This address is sent to the Memory Address Register. The Memory Data Register holds the data fetched from this address. This data is sent to the Current Instruction Register and the Control Unit decodes the instruction's opcode. The Program Counter is incremented.	5
------	--	----------

3(b)	1 mark for each shaded set of values <table><tr><th rowspan="2">Instruction address</th><th rowspan="2">ACC</th><th colspan="4">Memory address</th><th rowspan="2">IX</th><th rowspan="2">Output</th></tr><tr><th>365</th><th>366</th><th>367</th><th>368</th></tr><tr><td></td><td></td><td>1</td><td>3</td><td>65</td><td>66</td><td>0</td><td></td></tr><tr><td>200</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>201</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>202</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>203</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>204</td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>205</td><td></td><td></td><td></td><td></td><td></td><td>2</td><td></td></tr><tr><td>206</td><td>65</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>207</td><td></td><td></td><td></td><td></td><td></td><td></td><td>A</td></tr><tr><td>208</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>200</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>201</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>202</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>203</td><td>3</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>204</td><td></td><td>3</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>205</td><td></td><td></td><td></td><td></td><td></td><td>3</td><td></td></tr><tr><td>206</td><td>66</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>207</td><td></td><td></td><td></td><td></td><td></td><td></td><td>B</td></tr><tr><td>208</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>200</td><td>3</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>201</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>202</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>209</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Instruction address	ACC	Memory address				IX	Output	365	366	367	368			1	3	65	66	0		200	1							201								202								203	2							204		2						205						2		206	65							207							A	208								200	2							201								202								203	3							204		3						205						3		206	66							207							B	208								200	3							201								202								209								6
Instruction address	ACC			Memory address						IX	Output																																																																																																																																																																																											
		365	366	367	368																																																																																																																																																																																																	
		1	3	65	66	0																																																																																																																																																																																																
200	1																																																																																																																																																																																																					
201																																																																																																																																																																																																						
202																																																																																																																																																																																																						
203	2																																																																																																																																																																																																					
204		2																																																																																																																																																																																																				
205						2																																																																																																																																																																																																
206	65																																																																																																																																																																																																					
207							A																																																																																																																																																																																															
208																																																																																																																																																																																																						
200	2																																																																																																																																																																																																					
201																																																																																																																																																																																																						
202																																																																																																																																																																																																						
203	3																																																																																																																																																																																																					
204		3																																																																																																																																																																																																				
205						3																																																																																																																																																																																																
206	66																																																																																																																																																																																																					
207							B																																																																																																																																																																																															
208																																																																																																																																																																																																						
200	3																																																																																																																																																																																																					
201																																																																																																																																																																																																						
202																																																																																																																																																																																																						
209																																																																																																																																																																																																						
3(c)(i)	<table><tr><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	0	1	0	1	0	0	1																																																																																																																																																																																												
1	1	0	1	0	1	0	0																																																																																																																																																																																															
3(c)(ii)	1 mark for correct answer The number is divided by 8 (and only whole number retained)	1																																																																																																																																																																																																				